Physics Simulation for Computer Graphics

02 Fundamentals

Building Blocks of Numerical Simulation

Physics Model

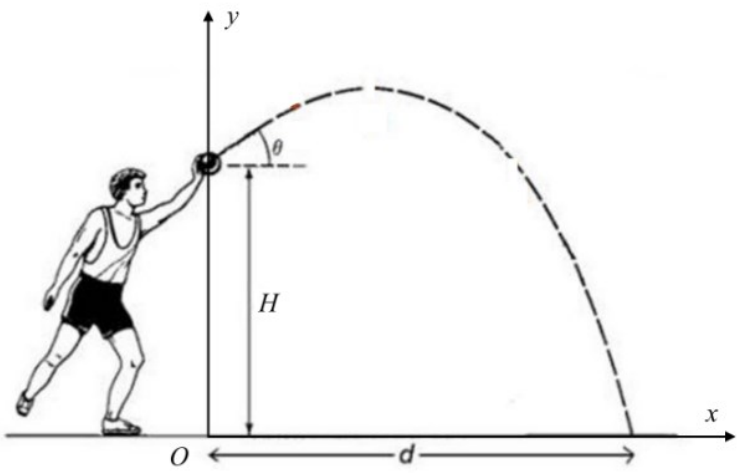
> The underlying physical laws necessary for the mathematical theory of a large part of physics and the whole of chemistry are thus completely known, and the difficulty is only that the exact application of these laws leads to equations much too complicated to be soluble.
>
> Paul M. Dirac, 1929

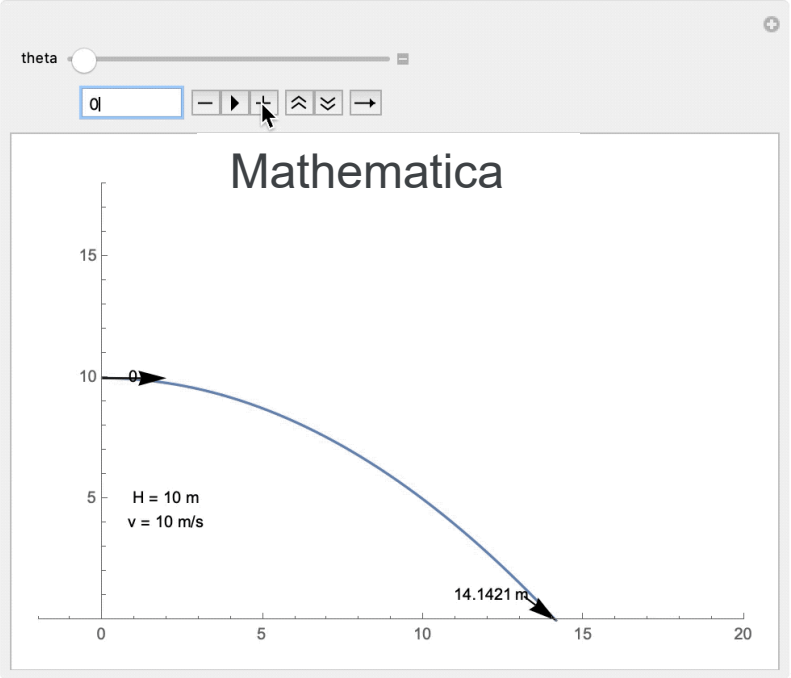
$$F = ma$$

- For a particle:

$$x = v_0 cos\theta \cdot t$$

$$y = H + v_0 sin\theta \cdot t + \frac{1}{2}(-g) \cdot t^2$$





Physics Model

> The underlying physical laws necessary for the mathematical theory of a large part of physics and the whole of chemistry are thus completely known, and the difficulty is only that the exact application of these laws leads to equations much too complicated to be soluble.
>
> Paul M. Dirac, 1929

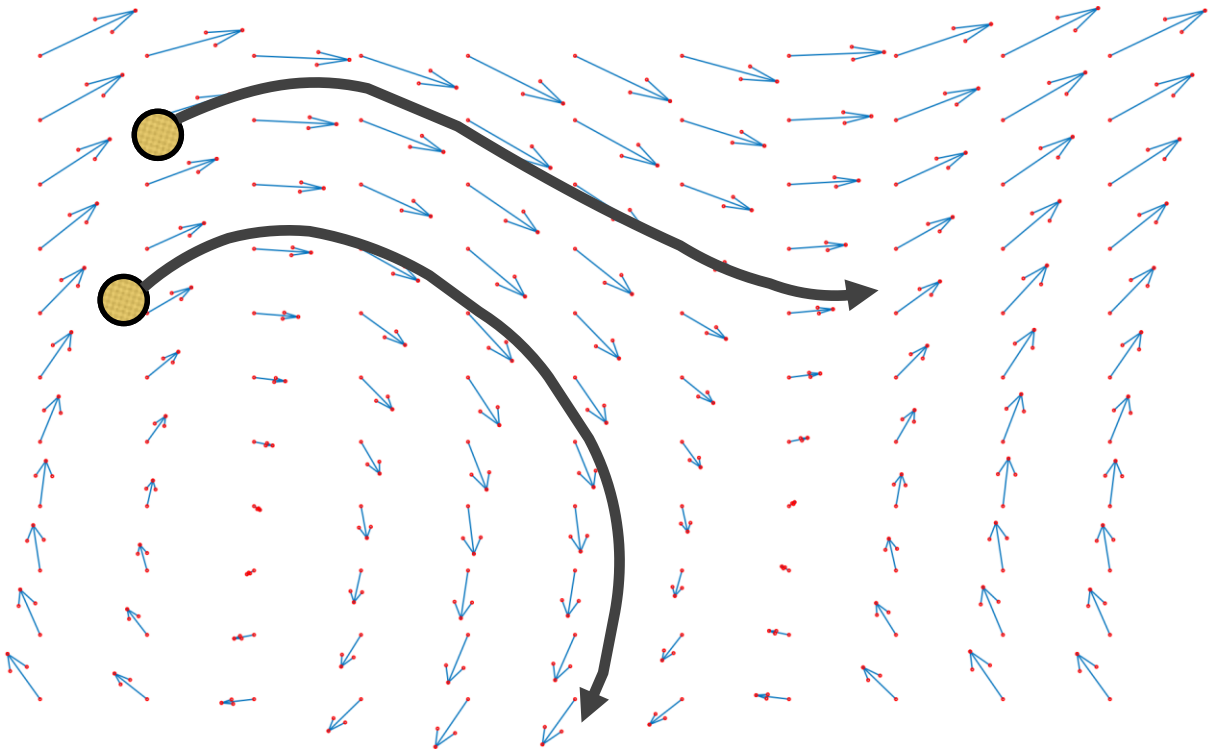
$$F = ma$$

- For a particle:
  - $a = f(t),\ v = \int f(t)\, dt + v_0$. Easy to solve!
- A piece of material consists of **infinite** particles
  - When **analytical solutions** don't exist, we look for **numerical solutions**
  - We need **discretization, calculus, differential equations, tensor analysis, field theories…**

Physics Model

- For a continuous field consisting of **infinite number** of particles :

Position $\boldsymbol{x}$ and velocity $\boldsymbol{v}$



- Kinematics: How things move?

$$\frac{d\mathbf{x}_p(t)}{dt} = \mathbf{v}(\mathbf{x}_p, t)$$

- Dynamics: Why things move?

$$\frac{d^2\mathbf{x}_p(t)}{dt^2} = \frac{\mathbf{f}(\mathbf{x}_p, t)}{m_p}$$

Building Blocks of a Simulator

Mathematical Models

$$\frac{d\mathbf{x}_p(t)}{dt} = \mathbf{v}(\mathbf{x}_p, t) \quad \& \quad \frac{d\mathbf{v}_p(t)}{dt} = \frac{\mathbf{f}(\mathbf{x}_p, t)}{m_p}$$

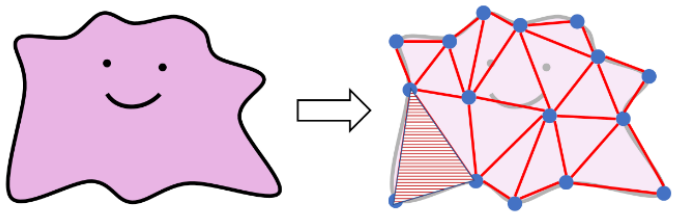
Spatial Discretization

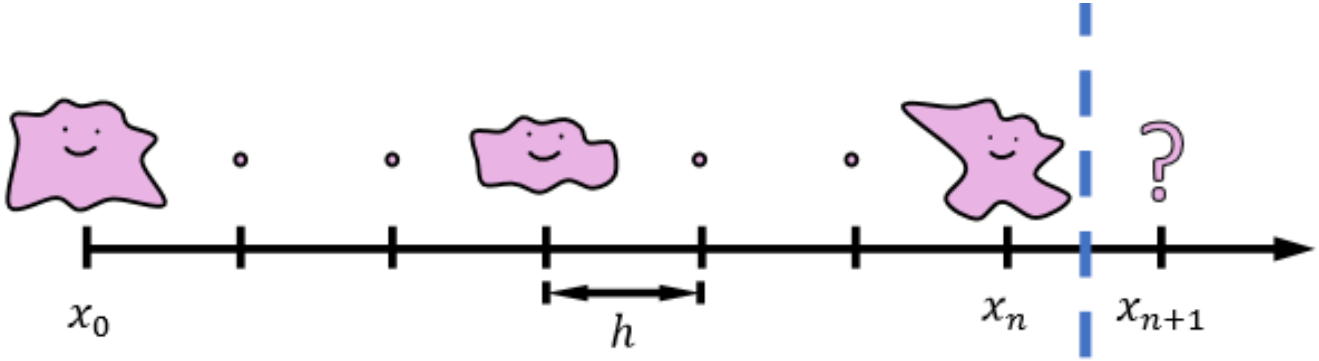
Define the State $(\mathbf{x}, \mathbf{v}, \mathbf{f}, \ldots)$

Temporal Discretization

Update the State $(\mathbf{x}, \mathbf{v}, \mathbf{f}, \ldots)$

Numerical Solver





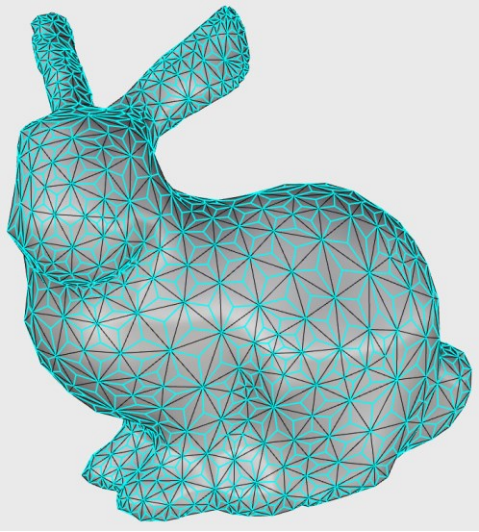
Physics Simulation

- Apply Appropriate *Spatial Discretization* and *Temporal Discretization*
- Start from some initial conditions (and boundary conditions,…)
- *Solve the temporal evolution according to mathematical rules*
  - Respect Physical Laws
  - Respect Constraints

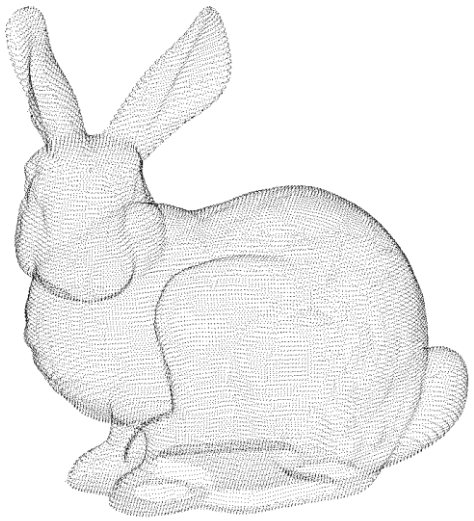
Spatial Discretization

Popular Spatial Representations

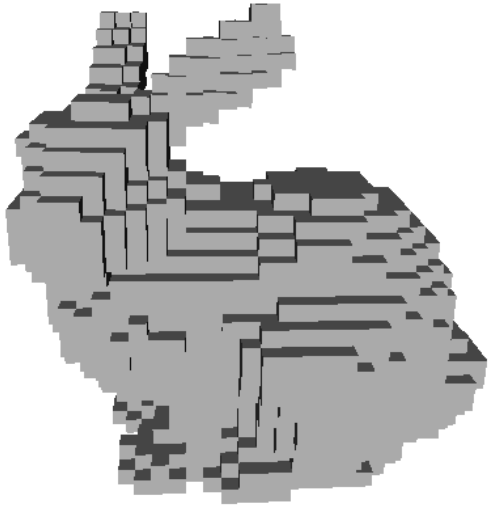
- In consistent to modeling, we have:



Triangle/Tetrahedra Meshes



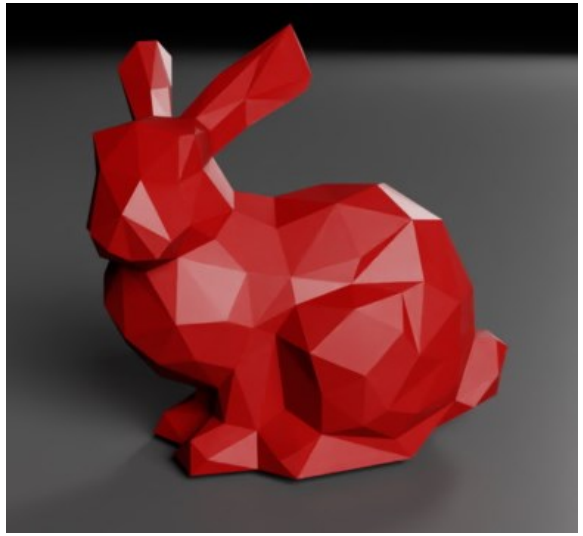
Particle Cloud



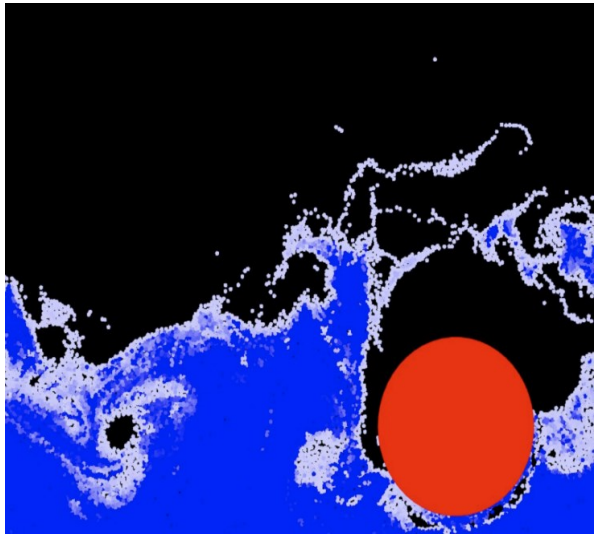
Volumetric Grid

Popular Spatial Representations

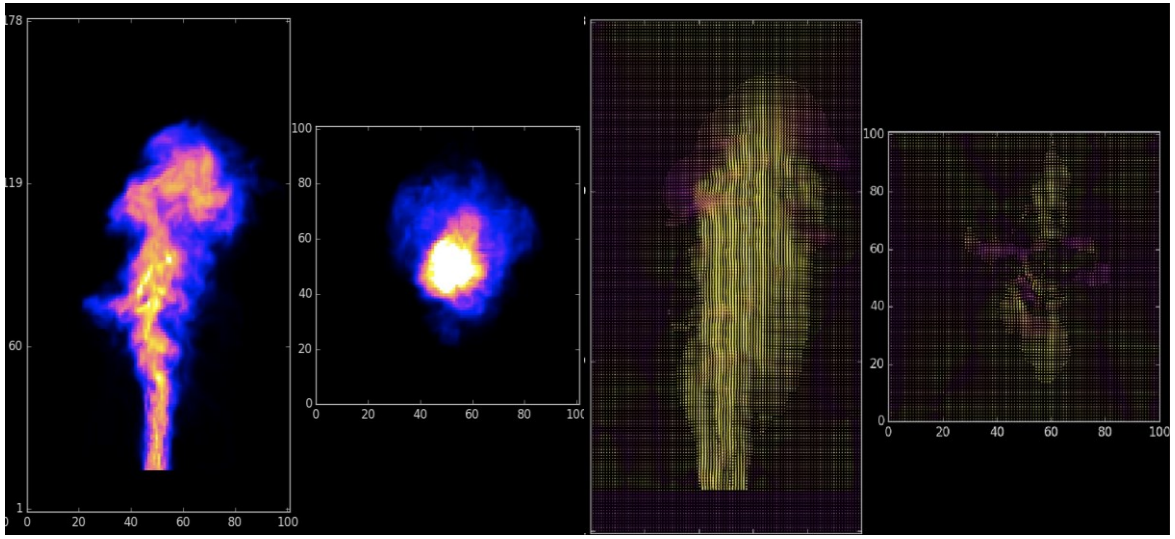
- In consistent to modeling, we have:



Triangle/Tetrahedra Meshes



Particle System



Volumetric Grid

Lagrangian viewpoint

Eulerian viewpoint

Two viewpoints

Lagrangian Viewpoint

Interested in $\dot{f}$,
usually ODE problems

Eulerian

Interested in $\frac{\partial f}{\partial t}$,
usually PDE problems

ODEs

- nth order ODE functions: $\mathbf{x}^{[n]} = f\left(t, \mathbf{x}, \dot{\mathbf{x}}, \ddot{\mathbf{x}}, \dots, \mathbf{x}^{[n-1]}\right)$

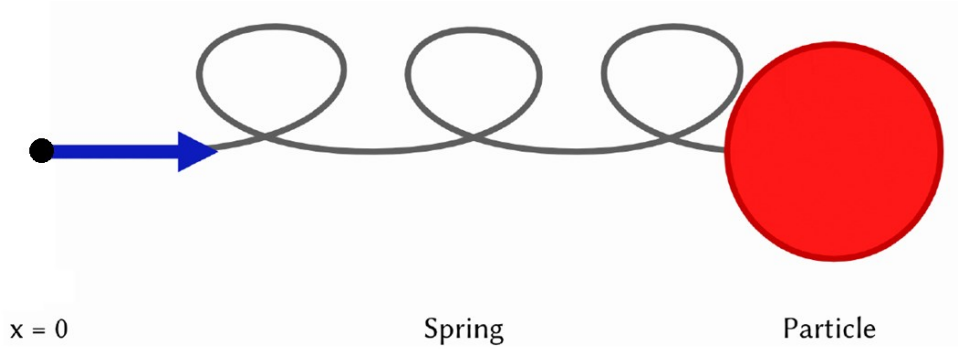
  defined by n initial conditions, i.e. $\mathbf{x}(t_0) = \mathbf{x}_0, \dot{\mathbf{x}}(t_0) = \dot{\mathbf{x}}_0, \dots, \mathbf{x}^{[n-1]}(t_0) = \mathbf{x}_0^{[n-1]}$

ODEs

- n^{th} order ODE functions: $\mathbf{x}^{[n]} = f\left(t, \mathbf{x}, \dot{\mathbf{x}}, \ddot{\mathbf{x}}, \dots, \mathbf{x}^{[n-1]}\right)$

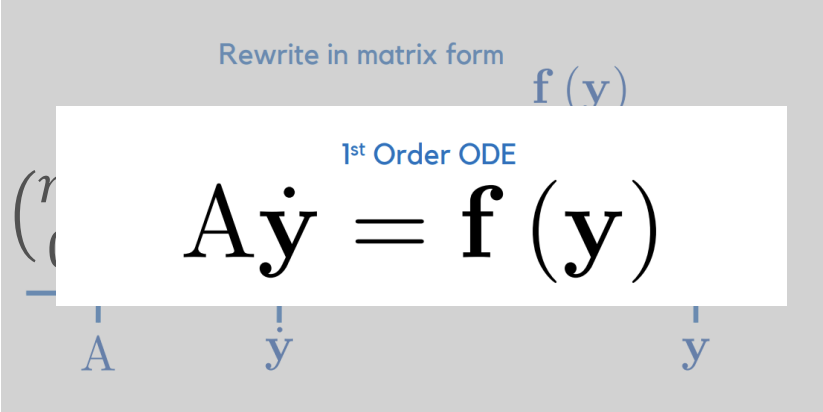
  defined by n initial conditions, i.e. $\mathbf{x}(t_0) = \mathbf{x}_0, \dot{\mathbf{x}}(t_0) = \dot{\mathbf{x}}_0, \dots, \mathbf{x}^{[n-1]}(t_0) = \mathbf{x}_0^{[n-1]}$

- Example:



$$m\frac{dv}{dt} = -kx$$

$$\frac{dx}{dt} = v$$

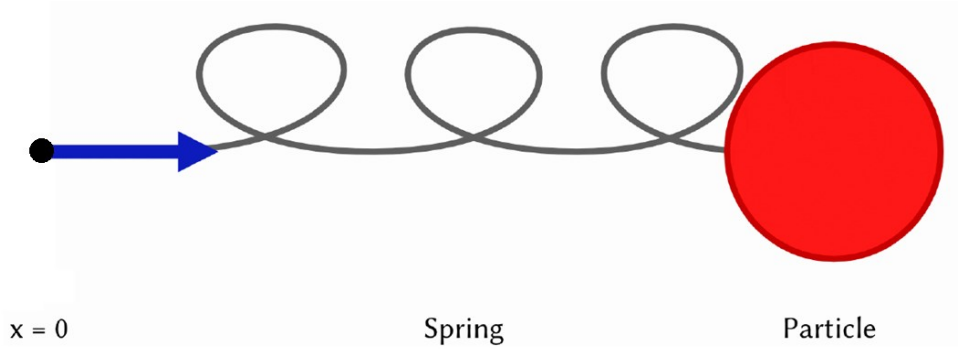


ODEs

- nth order ODE functions: $\mathbf{x}^{[n]} = f\left(t, \mathbf{x}, \dot{\mathbf{x}}, \ddot{\mathbf{x}}, \dots, \mathbf{x}^{[n-1]}\right)$

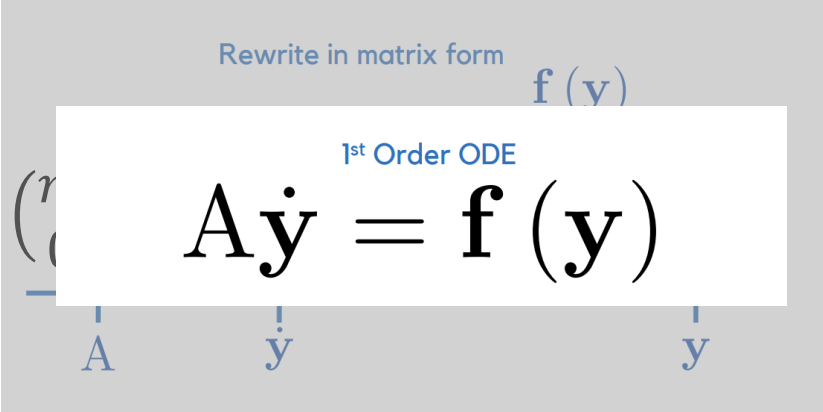
  defined by n initial conditions, i.e. $\mathbf{x}(t_0) = \mathbf{x}_0, \dot{\mathbf{x}}(t_0) = \dot{\mathbf{x}}_0, \dots, \mathbf{x}^{[n-1]}(t_0) = \mathbf{x}_0^{[n-1]}$

- Example:



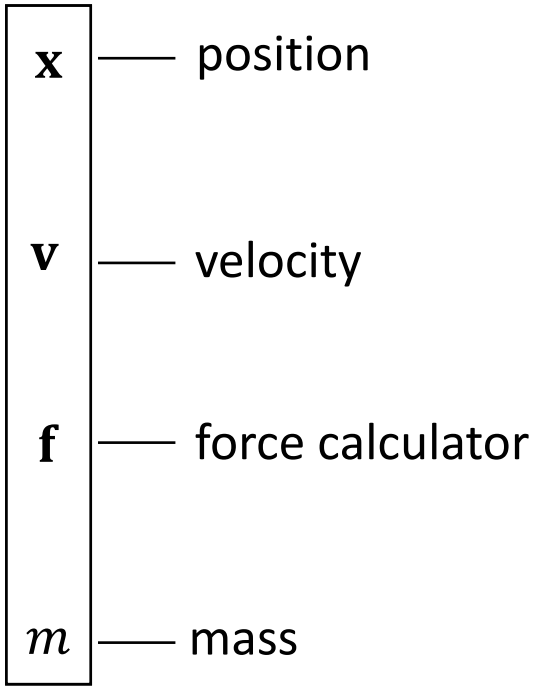
$$m\frac{dv}{dt} = -kx$$

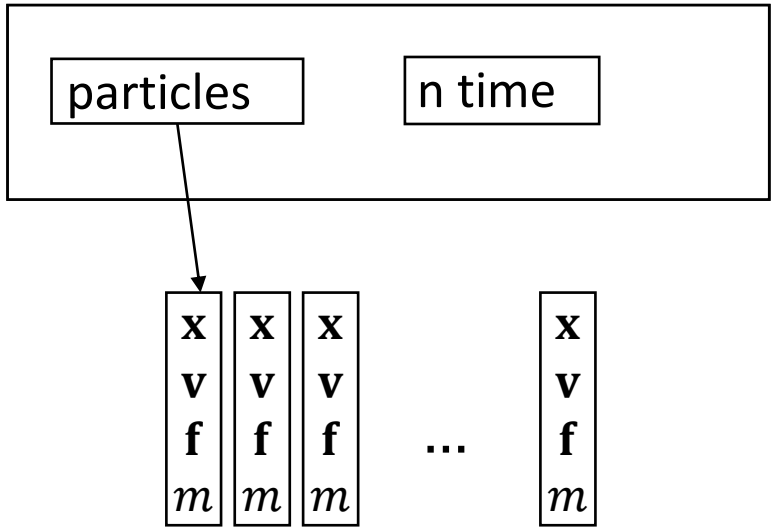
$$\frac{dx}{dt} = v$$



- ODE problems:
  - (common case) n initial conditions, $\{\mathbf{x}(t_0), \dot{\mathbf{x}}(t_0), \dots, \mathbf{x}^{[n-1]}(t_0)\}$ are given: initial value problem
  - (trickier case) conditions are distributed to (spatio-temporally) different states: boundary value problems

Spatial Discretization: Particle System





1st Order ODE

$$\mathrm{A}\dot{\mathbf{y}} = \mathbf{f}(\mathbf{y})$$

$$\begin{bmatrix} m & \cdots & & & & \\ \vdots & \ddots & \vdots & & & \\ & \cdots & m & & & \\ & & & 1 & \cdots & \\ & & & \vdots & \ddots & \vdots \\ & & & & \cdots & 1 \end{bmatrix} \frac{d}{dt} \begin{pmatrix} v \\ \vdots \\ v \\ \mathrm{x} \\ \vdots \\ \mathrm{x} \end{pmatrix} = f \begin{pmatrix} v \\ \vdots \\ v \\ \mathrm{x} \\ \vdots \\ \mathrm{x} \end{pmatrix}$$

Particle Structure **Particle System** **Particle System**

PDEs

- Partial differential equations (PDEs):

  derivatives with respect to different free variables, e.g. spatiotemporal variables, (x,y,z,t)

PDEs

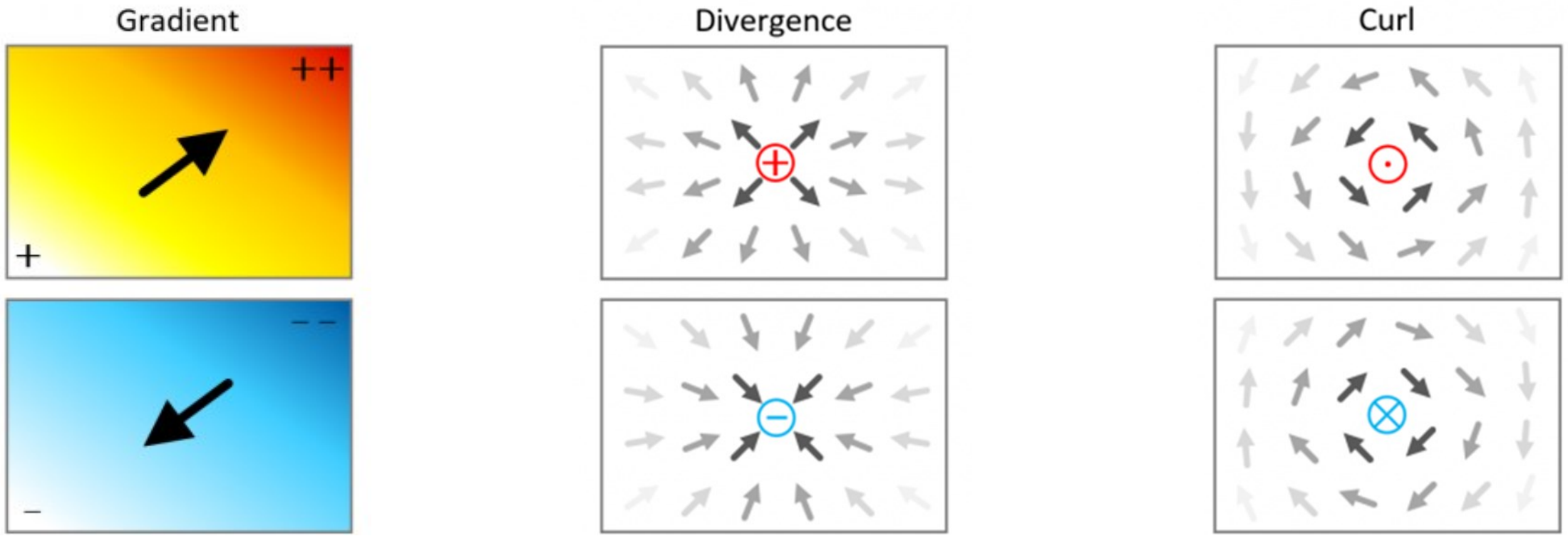
- Partial differential equations (PDEs):

  derivatives with respect to different free variables, e.g. spatiotemporal variables, (x,y,z,t)

- Example, diffusion equation (or heat equation)

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial {x_1}^2} + \frac{\partial^2 u}{\partial {x_2}^2} + \cdots + \frac{\partial^2 u}{\partial {x_n}^2} = \nabla \cdot \nabla u = \Delta u$$

Nabla Operator $\nabla : \left[\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z}\right]$



$$\nabla f(\mathbf{x}) = \frac{df}{d\mathbf{x}} = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}\right]$$

$f(\mathbf{x}) \in \mathbf{R}$,
Gradient $\in \mathbf{R}^3$

$$\nabla \cdot \mathbf{f}(\mathbf{x}) = \frac{d}{d\mathbf{x}} \cdot \mathbf{f} = \frac{\partial f}{\partial x} + \frac{\partial g}{\partial y} + \frac{\partial h}{\partial z}$$

$\mathbf{f}(\mathbf{x}) = (f, g, h) \in \mathbf{R}^3$,
Divergence $\in \mathbf{R}$

$$\nabla \times \mathbf{f}(\mathbf{x}) = \frac{d}{d\mathbf{x}} \times \mathbf{f} = \begin{bmatrix} \frac{\partial h}{\partial y} - \frac{\partial g}{\partial z} \\ \frac{\partial f}{\partial z} - \frac{\partial h}{\partial x} \\ \frac{\partial g}{\partial x} - \frac{\partial f}{\partial y} \end{bmatrix}$$

$\mathbf{f}(\mathbf{x}) = (f, g, h) \in \mathbf{R}^3$,
Curl $\in \mathbf{R}^3$

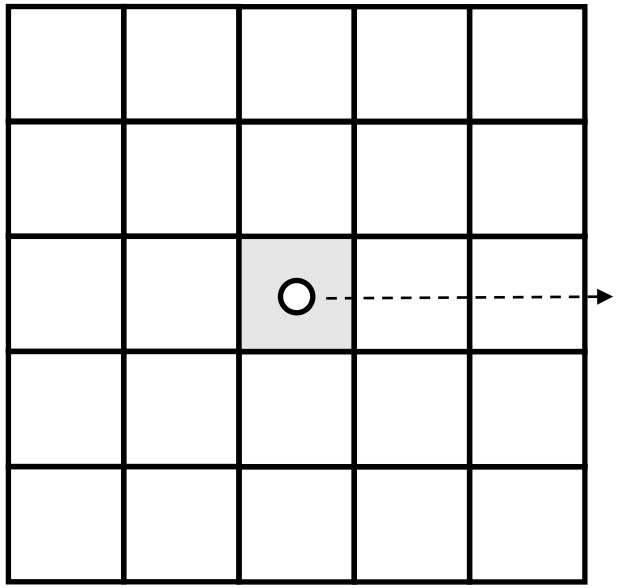
PDEs

- Partial differential equations (PDEs):

  derivatives with respect to different free variables, e.g. spatiotemporal variables, (x,y,z,t)

- Example, diffusion equation (or heat equation)

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial {x_1}^2} + \frac{\partial^2 u}{\partial {x_2}^2} + \cdots + \frac{\partial^2 u}{\partial {x_n}^2} = \nabla \cdot \nabla u = \Delta u$$



Regular Grid,

2D (or 3D) data arrays:

cells[D][H][W];

Structure:

- Scalars
  - Temperature
  - …

Given cells[D][H][W] at $t = 0$,

Calculate cells[D][H][W] for

$t_1 = h, t_2 = 2h, \ldots, t_n = nh$

with $\frac{u_{new} - u_{old}}{h} = \Delta u$

and boundary conditions

PDEs

- Partial differential equations (PDEs):

  derivatives with respect to different free variables, e.g. spatiotemporal variables, (x,y,z,t)

- Example, diffusion equation (or heat equation)

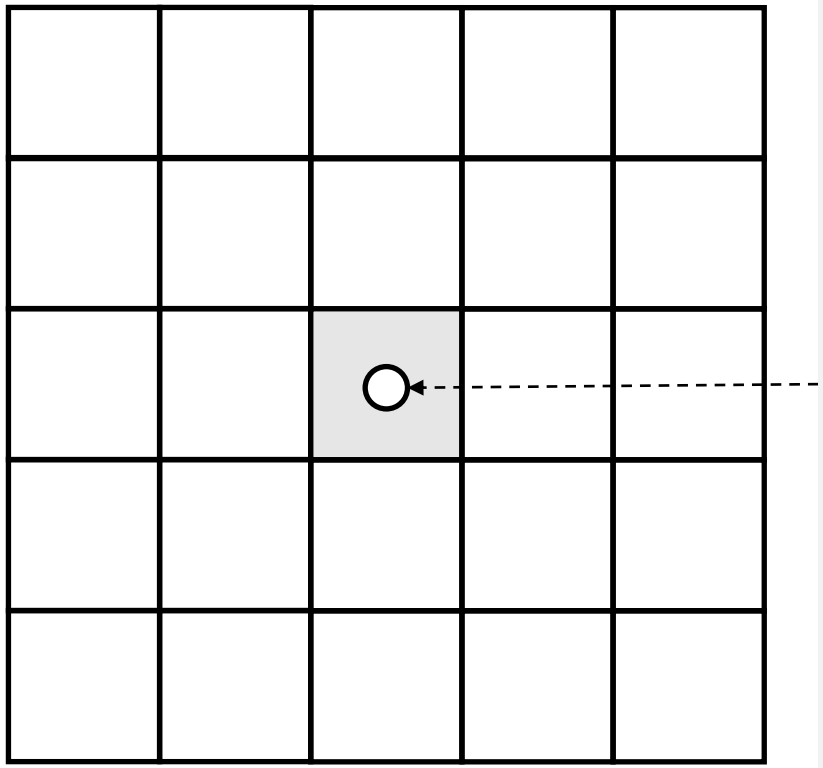
$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial {x_1}^2} + \frac{\partial^2 u}{\partial {x_2}^2} + \cdots + \frac{\partial^2 u}{\partial {x_n}^2} = \nabla \cdot \nabla u = \Delta u$$

- Boundary value problems are very important in PDE problems

- PDE solutions are defined by initial conditions and boundary conditions

Grid

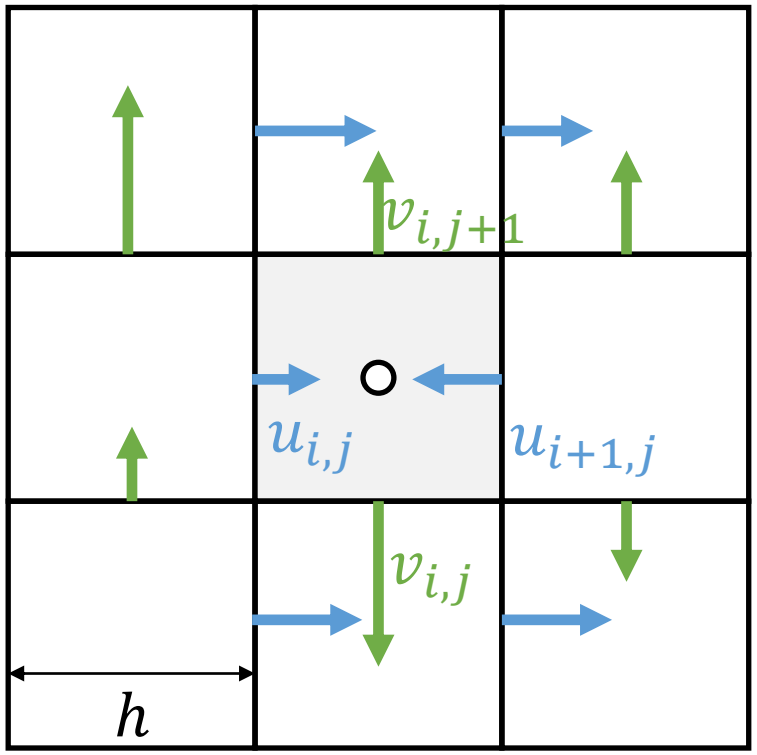
- Regular Grid



2D (or 3D) data arrays:

- Scalars
  - **Temperature**
  - Pressure
  - concentration
  - …
- Vectors
  - Velocities

- Staggered Grid



Two viewpoints

Lagrangian Viewpoint

Representation: Position $\mathbf{x}$ and velocity $\mathbf{v}$

Interested in $\dot{f}$, usually ODE problems:

Calculating $\frac{d\mathbf{x}}{dt}, \frac{d\mathbf{v}}{dt}$

And update $\mathbf{x}, \mathbf{v}$

Eulerian

Representation: (Mass) Distribution σ and vel $\mathbf{u}$

Interested in $\frac{\partial f}{\partial t}$, usually PDE problems:

Calculating $\frac{\partial \sigma}{\partial t}, \frac{\partial \boldsymbol{u}}{\partial t}$

And update the distribution σ and $\boldsymbol{u}$

Physics Simulation

- Apply Appropriate *Spatial Discretization* and *Temporal Discretization*
- Start from some initial conditions (and boundary conditions,…)
- *Solve the temporal evolution according to mathematical rules*
  - Respect Physical Laws
  - Respect Constraints

Popular Spatial Representations

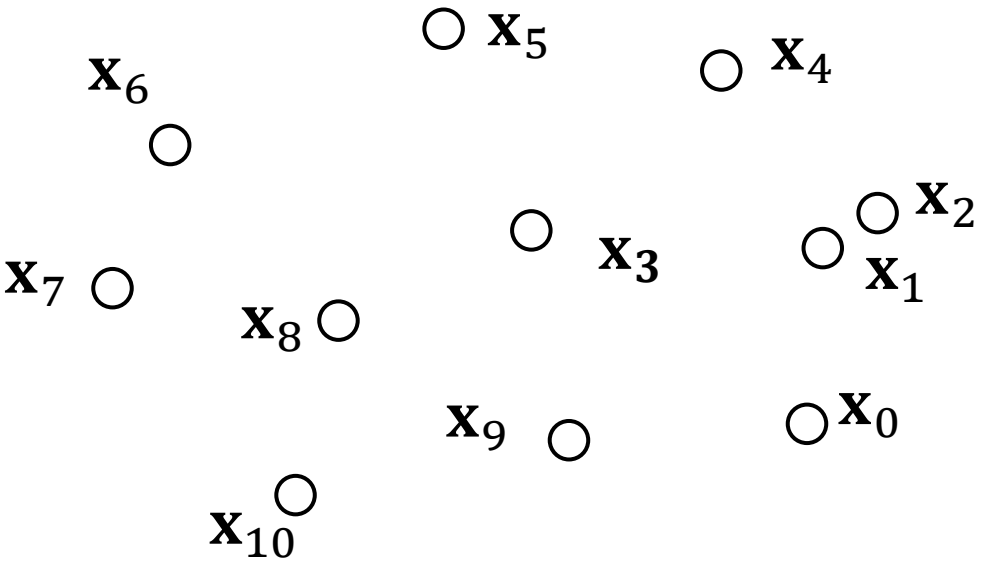
Particles	Mesh	Grid
Advantages ❖ Simple ❖ Easy to map to previous points in time	**Advantages** ❖ Easy to map to previous points in time ❖ Can conform to boundaries	**Advantages** ❖ Simple ❖ Can take advantage of structure for efficiency
Disadvantages ❖ Difficult to perform integration because they don't partition space	**Disadvantages** ❖ Difficult to generate meshes ❖ Re-meshing	**Disadvantages** ❖ Difficult to track shape over time ❖ Difficult to handle non-grid-aligned boundaries

Hybrid	Advantages	Disadvantages
	❖ Advantages of underlying structures	❖ Complexity; Computational and accuracy costs from mapping between structures

Representations,
Vector and Tensor Calculus

Particle System

$\mathbf{x}, \mathbf{v}$: Not geometric vectors, stacked vectors.
$\mathbf{m}$: stacked scalar, $\mathbf{M}$: diagonal matrix



represented by

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_0 \\ \vdots \\ \mathbf{x}_i \\ \vdots \\ \mathbf{x}_{10} \end{bmatrix} \in \mathbf{R}^{33} \qquad \mathbf{v} = \begin{bmatrix} \mathbf{v}_0 \\ \vdots \\ \mathbf{v}_i \\ \vdots \\ \mathbf{v}_{10} \end{bmatrix} \in \mathbf{R}^{33}$$

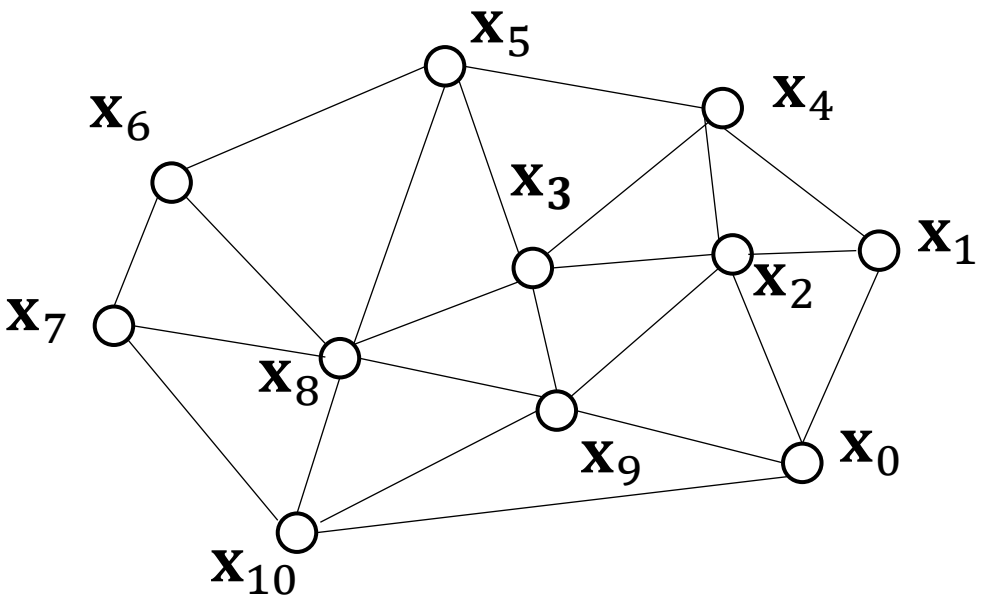
$$\mathrm{m} = \begin{bmatrix} \mathrm{m}_0 \\ \vdots \\ \mathrm{m}_i \\ \vdots \\ \mathrm{m}_{10} \end{bmatrix}, \qquad \mathbf{M} = \begin{bmatrix} \mathrm{m}_0 & & & & \\ & \ddots & & \mathbf{0} & \\ & & \mathrm{m}_i & & \\ & \mathbf{0} & & \ddots & \\ & & & & \mathrm{m}_{10} \end{bmatrix}$$

Particle Mass, Positions and Velocities: $\mathbf{m}, \mathbf{x}, \mathbf{v}$

$$\mathbf{f} = m\mathbf{a} \quad \Rightarrow \quad \mathbf{f} = \mathrm{M}\frac{d\mathbf{v}}{dt} \quad \text{and} \quad \frac{d\mathbf{v}}{dt} = \mathrm{M}^{-1}\,\mathbf{f}$$

Neighbor Particles: Grid Hash Table, KD-Tree, ⋯

Mesh



represented by

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_0 \\ \vdots \\ \mathbf{x}_i \\ \vdots \\ \mathbf{x}_{10} \end{bmatrix} \in \mathbf{R}^{33} \qquad \mathbf{v} = \begin{bmatrix} \mathbf{v}_0 \\ \vdots \\ \mathbf{v}_i \\ \vdots \\ \mathbf{v}_{10} \end{bmatrix} \in \mathbf{R}^{33}$$

Vertex Positions and Velocities $\mathbf{x}, \mathbf{v}$

Triangle list: {(0,1,2),(2,1,4),(2,4,3), ⋯ } (index triples)

Triangle Edge list: {{0,1,0}, {1,2,0}, {2,0,0}, {1,2,1}, {1,4,1}, {2,4,1}, {2,4,2}⋯}

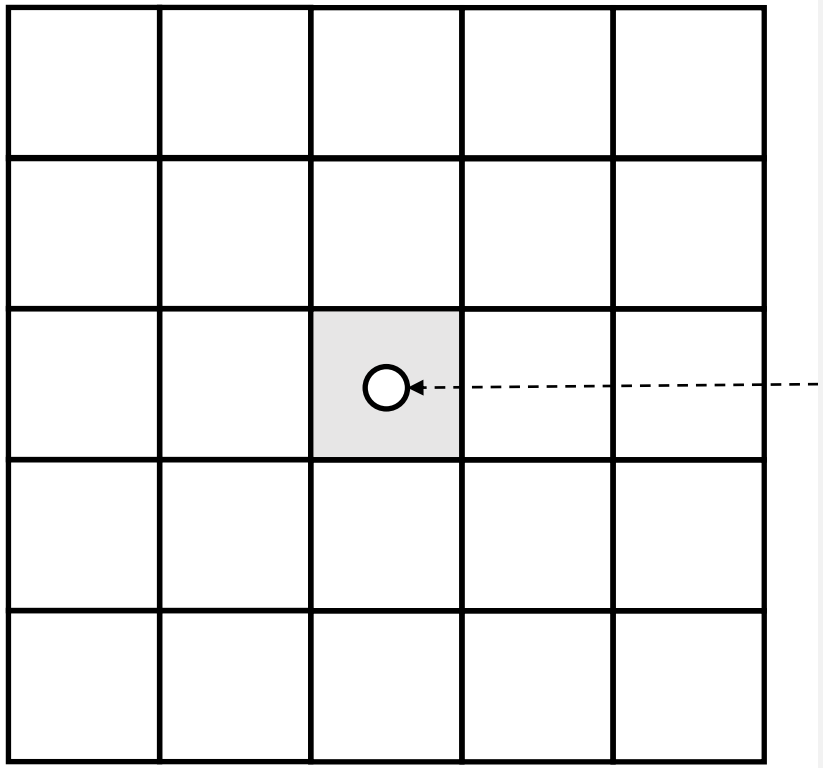
Sort and remove

Edge list: {{0,1}, {0,2}, {0,9}, {0,10}, {1,2}, {1,3}, {1,4},⋯}

Neighboring triangle list: {{0, 1}, {1, 2}}

Grid

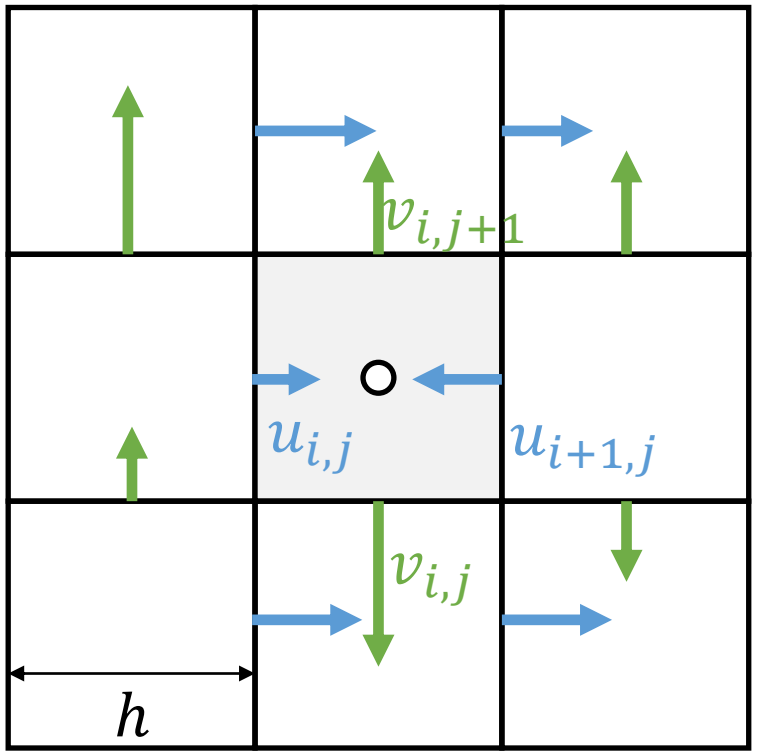
- Regular Grid



2D (or 3D) data arrays:

- Scalars
  - Density/color
  - Pressure
  - Temperature
  - …
- Vectors
  - Velocities

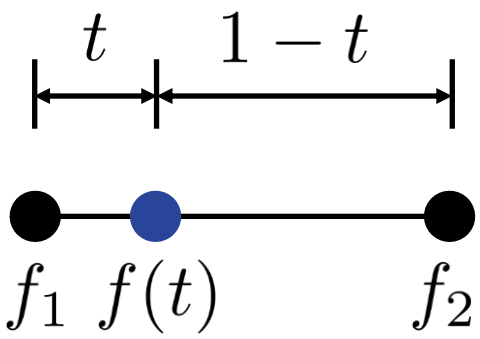
- Staggered Grid



Interpolation

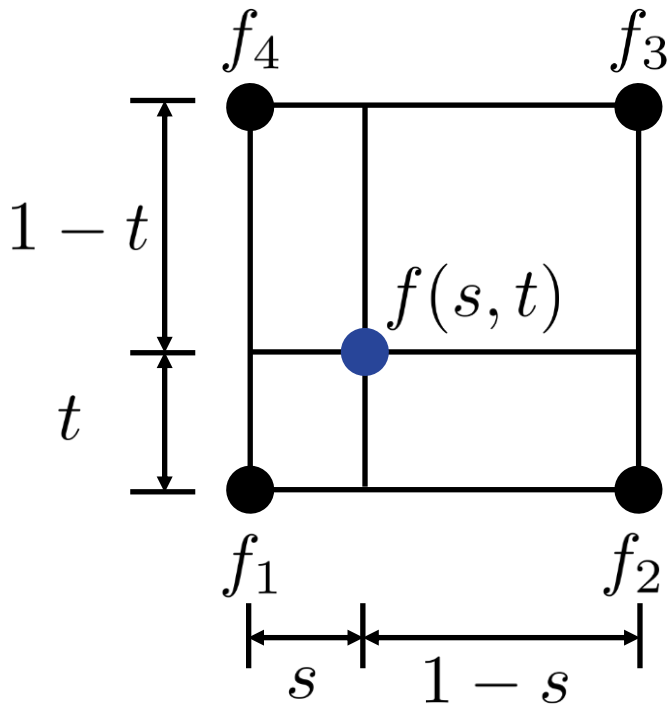
- Linear Interpolation (1D)

$$f(t) = (1-t)f_1 + tf_2.$$



- Bilinear Interpolation (2D)

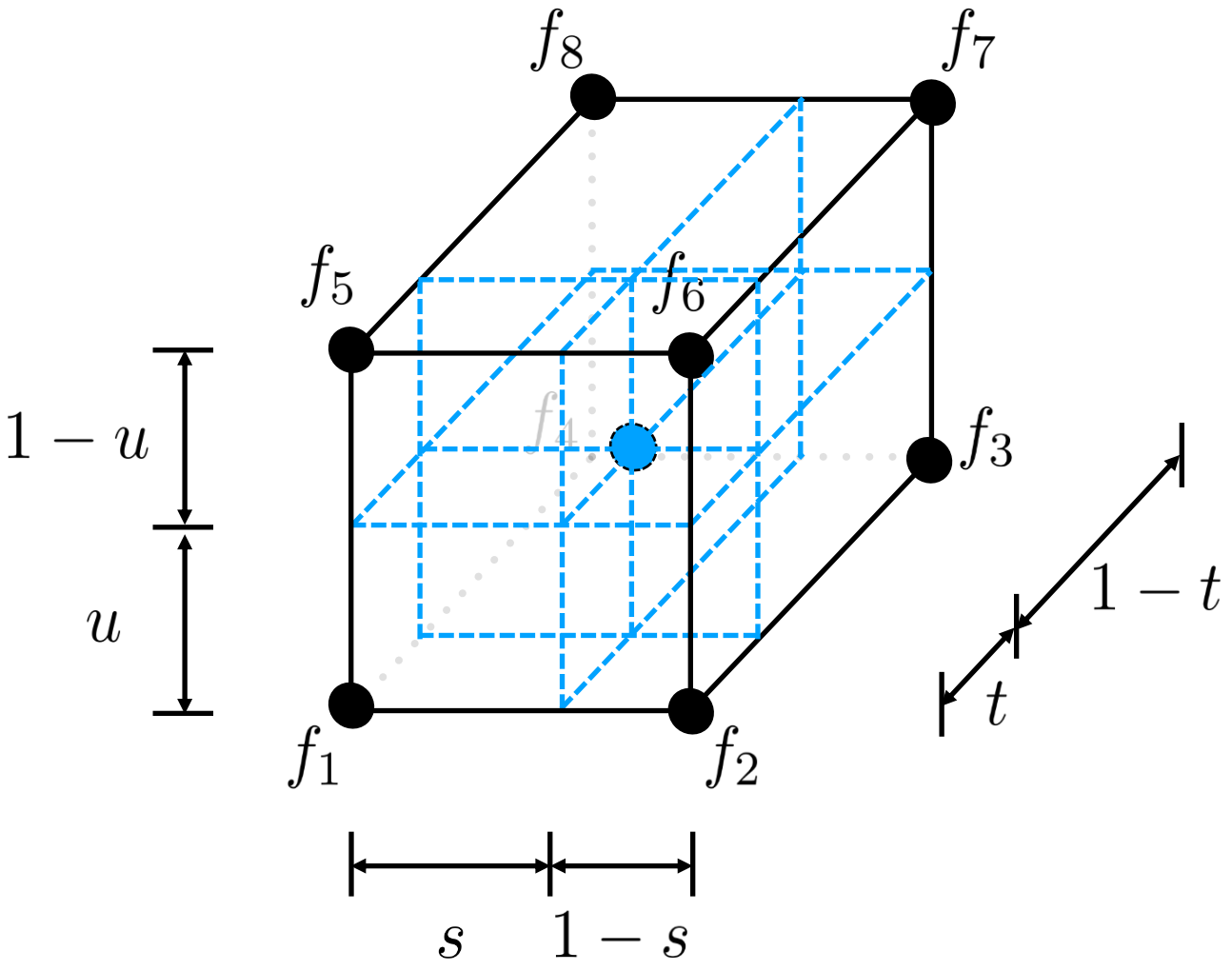
$$f(s,t) = (1-s)(1-t)f_1 + s(1-t)f_2 + stf_3 + (1-s)tf_4$$



Interpolation

- Trilinear Interpolation (3D)

$$
\begin{aligned}
&f(s,t,u) = \\
&(1-s)(1-t)(1-u)f_1 \\
&+ s(1-t)(1-u)f_2 \\
&+ st(1-u)f_3 \\
&+ (1-s)t(1-u)f_4 \\
&+ (1-s)(1-t)uf_5 \\
&+ s(1-t)uf_6 \\
&+ stuf_7 \\
&+ (1-s)tuf_8
\end{aligned}
$$



Barycentric Coordinates

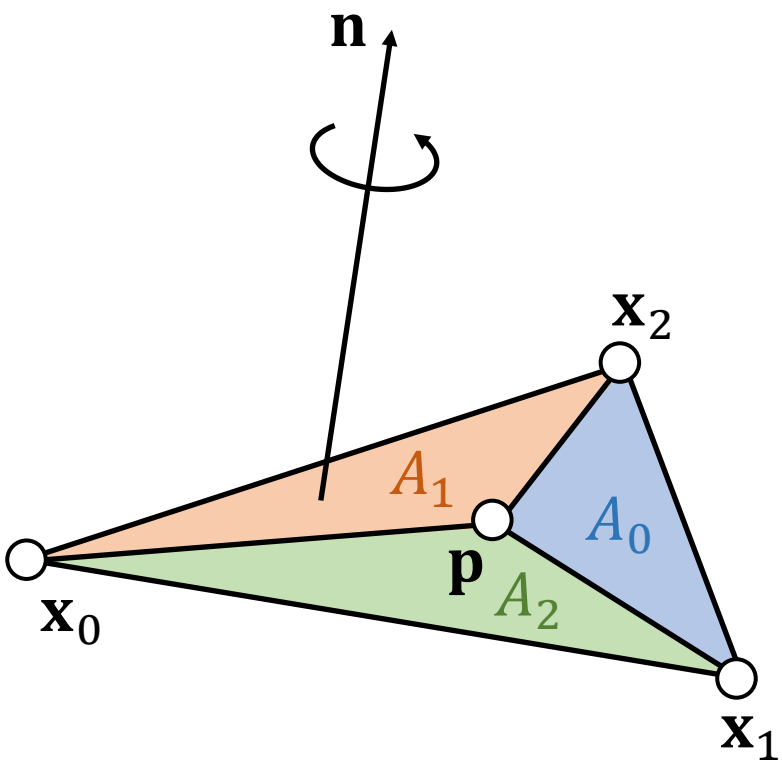
$$\boxed{\mathbf{p} = b_0\mathbf{x}_0 + b_1\mathbf{x}_1 + b_2\mathbf{x}_2}$$

$$\left.\begin{aligned} b_0 &= A_0/A \\ b_1 &= A_1/A \\ b_2 &= A_2/A \end{aligned}\right\} \quad b_0 + b_1 + b_2 = 1$$

$$A_0 = \tfrac{1}{2}(\mathbf{x}_1 - \mathbf{p}) \times (\mathbf{x}_2 - \mathbf{p}) \cdot \mathbf{n}$$

$$A_1 = \tfrac{1}{2}(\mathbf{x}_2 - \mathbf{p}) \times (\mathbf{x}_0 - \mathbf{p}) \cdot \mathbf{n}$$

$$A_2 = \tfrac{1}{2}(\mathbf{x}_0 - \mathbf{p}) \times (\mathbf{x}_1 - \mathbf{p}) \cdot \mathbf{n}$$

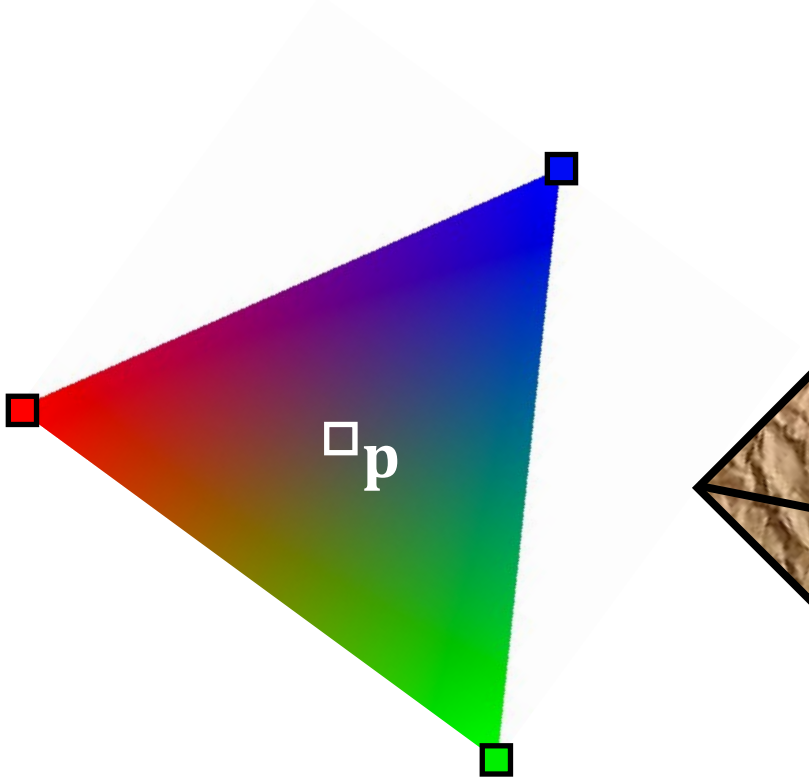


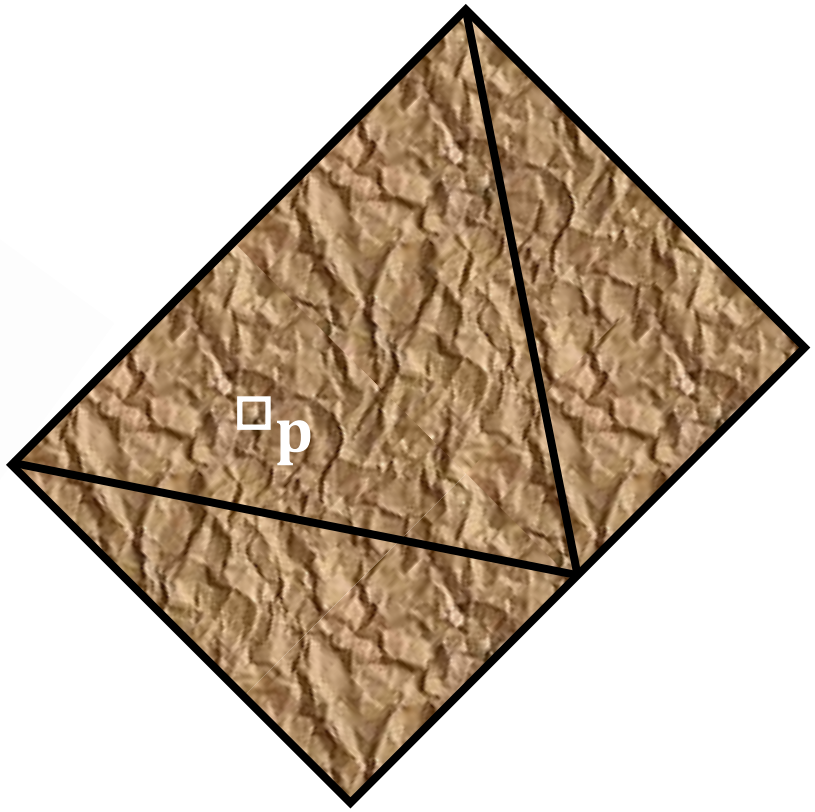
Inside: $0 < b_i < 1 \;\; (i = 0,1,2)$, and coplanar

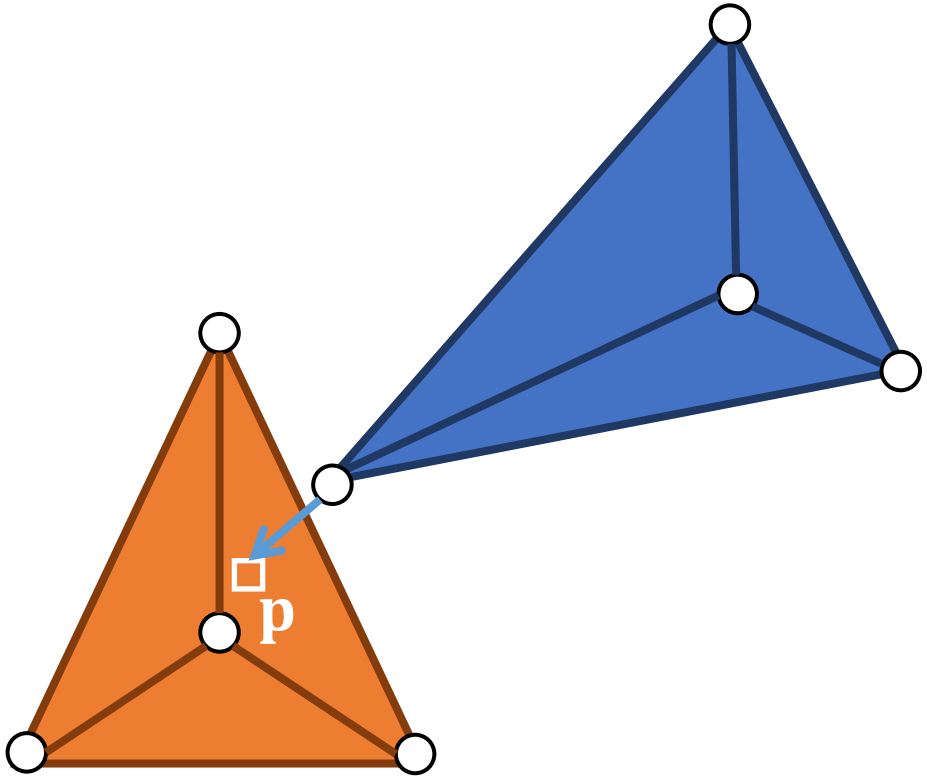
Outside: otherwise

Barycentric Coordinates

- Gouraud Shading, Texture Mapping, Collision Detection,…







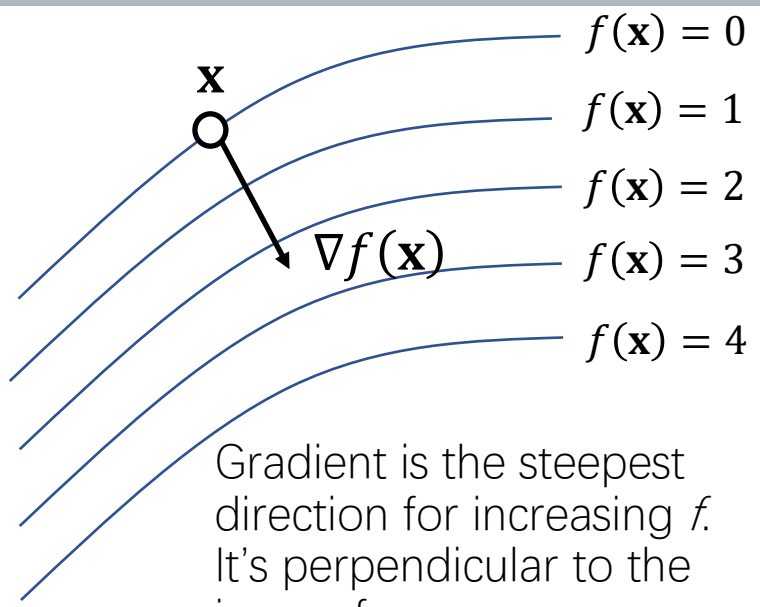
1st-Order Derivatives

- If x and $f(x) \in \mathbf{R}$, then $df = \frac{df}{dx}dx$. $f'(x) = \frac{df}{dx}$, $f'(x) \in \mathbf{R}$
- If $f(\mathbf{x}) \in \mathbf{R}$, then

$$df = \frac{\partial f}{\partial x}dx + \frac{\partial f}{\partial y}dy + \frac{\partial f}{\partial z}dz = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} & \frac{\partial f}{\partial z} \end{bmatrix} \begin{bmatrix} dx \\ dy \\ dz \end{bmatrix} = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \\ \partial f/\partial z \end{bmatrix} \cdot \begin{bmatrix} dx \\ dy \\ dz \end{bmatrix}.$$

Gradient

$$\nabla f(\mathbf{x}) = \frac{df}{d\mathbf{x}} = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}\right]$$



Gradient is the steepest direction for increasing *f*. It's perpendicular to the iso-surface.

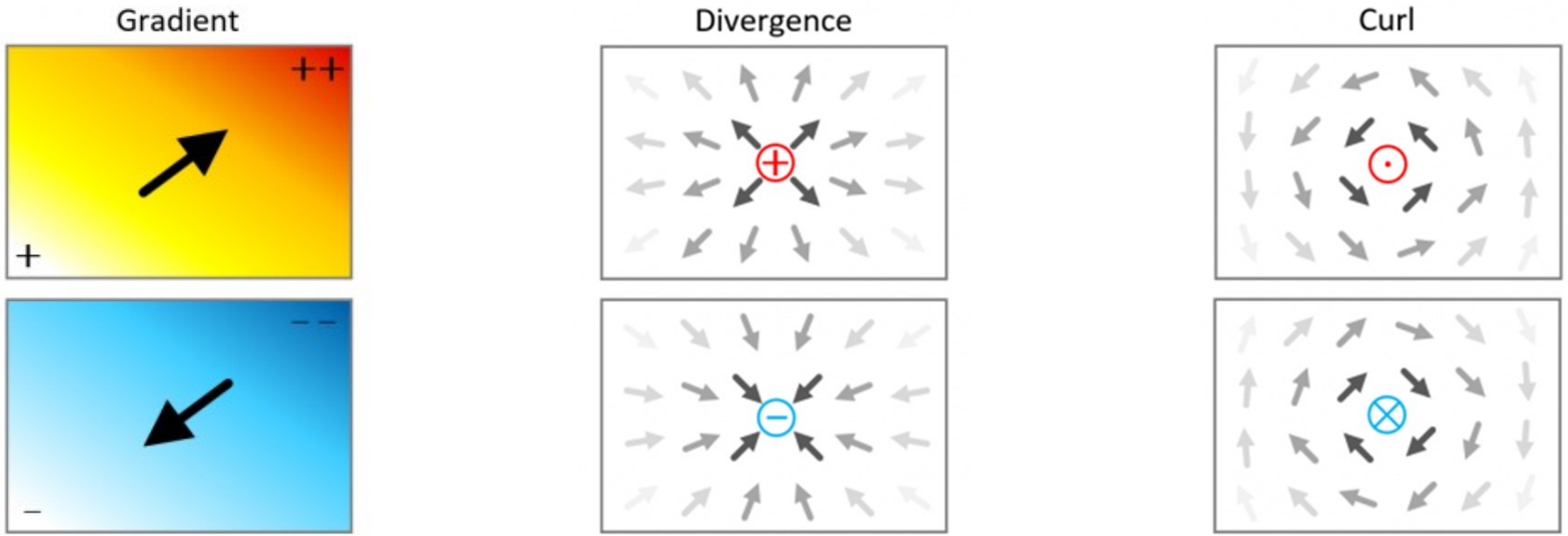
- If $\mathbf{f}(\mathbf{x}) = \begin{bmatrix} f(\mathbf{x}) \\ g(\mathbf{x}) \\ h(\mathbf{x}) \end{bmatrix} \in \mathbf{R}^3$, then:

$$d\mathbf{f} = \frac{\partial \mathbf{f}}{\partial x}dx + \frac{\partial \mathbf{f}}{\partial y}dy + \frac{\partial \mathbf{f}}{\partial z}dz$$
$$= \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x} & \frac{\partial \mathbf{f}}{\partial y} & \frac{\partial \mathbf{f}}{\partial z} \end{bmatrix} \begin{bmatrix} dx \\ dy \\ dz \end{bmatrix}.$$

Jacobian

$$\mathbf{J}(\mathbf{f}) = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x} & \frac{\partial \mathbf{f}}{\partial y} & \frac{\partial \mathbf{f}}{\partial z} \end{bmatrix} = \begin{bmatrix} \nabla^{\mathrm{T}} f \\ \nabla^{\mathrm{T}} g \\ \nabla^{\mathrm{T}} h \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} & \frac{\partial f}{\partial z} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} & \frac{\partial g}{\partial z} \\ \frac{\partial h}{\partial x} & \frac{\partial h}{\partial y} & \frac{\partial h}{\partial z} \end{bmatrix}$$

Nabla Operator $\nabla : \left[\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z}\right]$



$$\nabla f(\mathbf{x}) = \frac{df}{d\mathbf{x}} = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}\right]$$

$f(\mathbf{x}) \in \mathbf{R}$,
Gradient $\in \mathbf{R}^3$

$$\nabla \cdot \mathbf{f}(\mathbf{x}) = \frac{d}{d\mathbf{x}} \cdot \mathbf{f} = \frac{\partial f}{\partial x} + \frac{\partial g}{\partial y} + \frac{\partial h}{\partial z}$$

$\mathbf{f}(\mathbf{x}) = (f, g, h) \in \mathbf{R}^3$,
Divergence $\in \mathbf{R}$

$$\nabla \times \mathbf{f}(\mathbf{x}) = \frac{d}{d\mathbf{x}} \times \mathbf{f} = \begin{bmatrix} \frac{\partial h}{\partial y} - \frac{\partial g}{\partial z} \\ \frac{\partial f}{\partial z} - \frac{\partial h}{\partial x} \\ \frac{\partial g}{\partial x} - \frac{\partial f}{\partial y} \end{bmatrix}$$

$\mathbf{f}(\mathbf{x}) = (f, g, h) \in \mathbf{R}^3$,
Curl $\in \mathbf{R}^3$

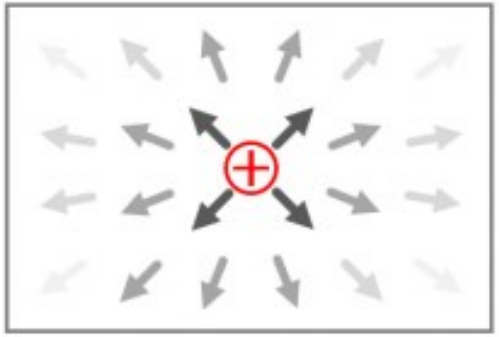
2nd-Order Derivatives

Laplacian: Divergence of Gradient

If $f(\mathbf{x}) \in \mathbf{R}$, then:

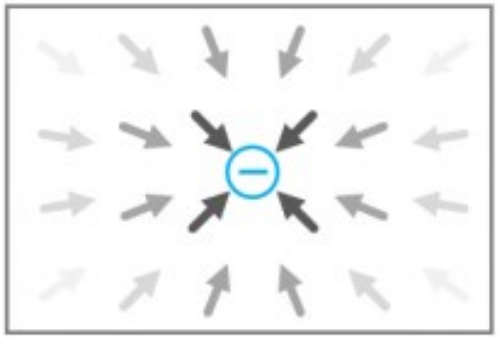
Gradient

$$\nabla f(\mathbf{x}) = \frac{\partial f}{\partial \mathbf{x}} = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}\right]$$



Hessian

$$\mathbf{H} = \mathbf{J}(\nabla f(\mathbf{x})) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial x \partial z} \\ \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial y^2} & \frac{\partial^2 f}{\partial y \partial z} \\ \frac{\partial^2 f}{\partial x \partial z} & \frac{\partial^2 f}{\partial y \partial z} & \frac{\partial^2 f}{\partial z^2} \end{bmatrix}$$



$$\frac{\partial u}{\partial t} = \nabla \cdot \nabla u = \Delta u$$

Laplacian ($\Delta = \nabla^2 = \nabla \cdot \nabla$)

$$\nabla \cdot \nabla f(\mathbf{x}) = \left[\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z}\right] \cdot \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}\right] = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2} = \text{trace(H)}$$

Fundamentals

- Backgrounds: Computer Graphics =Modeling + **Simulation** & Animation + Rendering
- Physics Models: ODE & PDE functions (Physical Laws + Geometric Constraints)

Nth-order ODEs:	**Nth-order PDEs**
$\{\mathbf{s}(\mathrm{t}): \mathrm{T} \to \mathbb{R}^d, f\left(\mathbf{s}, \dot{\mathbf{s}}, \dots, \mathbf{s}^{[n]}\right) = 0\}$	$\{\mathbf{s}(\mathbf{x}, \mathrm{t}): \Omega \times \mathrm{T} \to \mathbb{R}^d, f\left(\mathbf{s}, \frac{\partial s}{\partial t}, \dots, \nabla \mathbf{s}, \nabla^2 \mathbf{s}, \dots\right) = 0\}$
requires n initial conditions $\{\mathbf{s}_0, \dot{\mathbf{s}}_0, \dots, \mathbf{s}_0^{[n-1]}\}$ to solve	requires $\left\{\mathbf{s}_0, \frac{\partial s_0}{\partial t}, \dots\right\}$ and BC $\left\{\mathbf{s}_t, \frac{\partial \mathbf{s}_0}{\partial t}, \dots\right\}_{\partial\Omega}$ to solve

$h(\mathbf{s}, \dots) = 0$ and/or $g(\mathbf{s}, \dots) \geq 0$

- Basic Building Blocks of Numerical Simulator:

Spatial Discretization	Time Integration	Numerical Solver
• **Perspectives:** Lagrangian & Eulerian • Popular **Representation**: Mesh, Grid, Particle, Hybrid, ⋯ ➢ Degrees of Freedom (DoFs) ➢ Interpolation ➢ Differential Operators ➢ Spatial Integration	• **Explicit & Implicit Methods** Explicit: $\mathbf{s}_{t+1} = f(\mathbf{s}_t)$ Implicit: $\mathbf{s}_{t+1} = f_1(\mathbf{s}_t) + f_2(\mathbf{s}_{t+1})$ $\Rightarrow \mathbf{s}_{t+1} = argmin_{\mathbf{s}}\, E(\mathbf{s})$ • **Evaluation**: (Stability, Speed) + (Consistency, Accuracy)	• **Linear System** $A\mathbf{s} = b$ (*e.g., finite difference for Fluids*) LU, Cholesky, Conjugate Gradient, PCG • **Nonlinear Optimization** (*e.g. FEM for Soft Body*) $f(\mathbf{s}, \mathbf{s}^n, \lvert\mathbf{s}\rvert, \dots) = 0 \Leftrightarrow \mathrm{argmin}\ \mathbf{E}(\mathbf{s})$ where $d\mathbf{E}(\mathbf{s})/d\mathbf{s} = f(\mathbf{s})$ Newton's method, Quasi-Newton, Projective Dynamics, BFGS, ⋯ • **Optimization with Constraints** (*e.g. Incremental Potential Contact*) $\mathrm{argmin}\ \mathbf{E}(\mathbf{s})\ \mathrm{s.t.}\ g(\mathbf{s}) \geq 0\ \mathrm{and/or}\ h(\mathbf{s}) = 0$ penalty methods, position-based methods, projection corrections, constrained dynamics (*Lagrange Multipliers, Primal-Dual methods, ADMM*) ⋯

Time Integrations

Building Blocks of a Simulator

Mathematical Models

$$\frac{d\mathbf{x}_p(t)}{dt} = \mathbf{v}(\mathbf{x}_p, t) \quad \& \quad \frac{d\mathbf{v}_p(t)}{dt} = \frac{\mathbf{f}(\mathbf{x}_p, t)}{m_p}$$

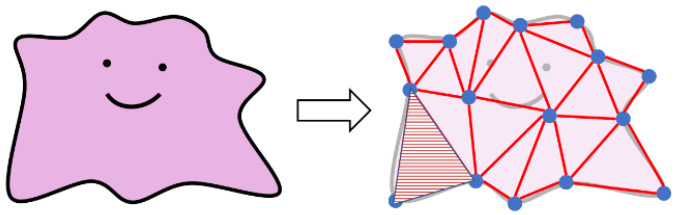
Spatial Discretization

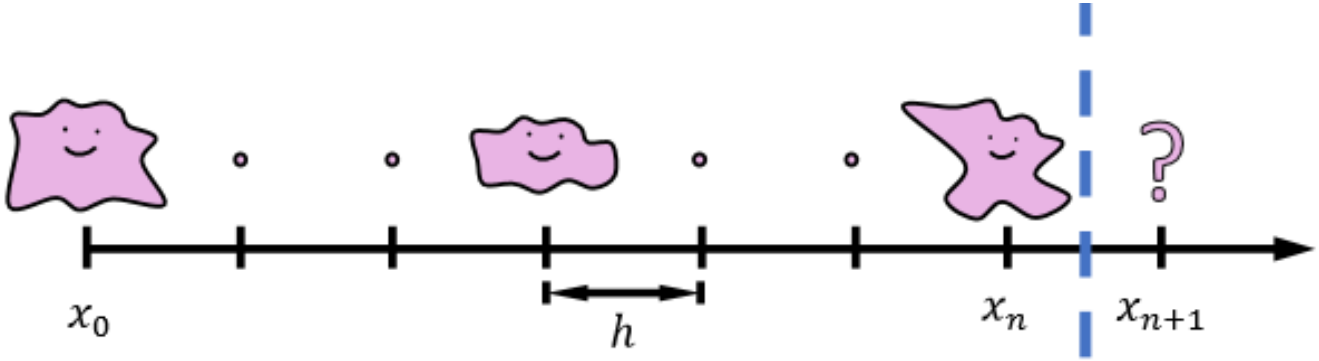
Define the State $(\mathbf{x}, \mathbf{v}, \mathbf{f}, \ldots)$

Temporal Discretization

Update the State $(\mathbf{x}, \mathbf{v}, \mathbf{f}, \ldots)$

Numerical Solver





Temporal Integration

$$\frac{d\mathbf{x}_p(t)}{dt} = \mathbf{v}(\mathbf{x}_p, t)$$

$$\frac{d\mathbf{v}_p(t)}{dt} = \frac{\mathbf{f}(\mathbf{x}_p, t)}{m_p}$$

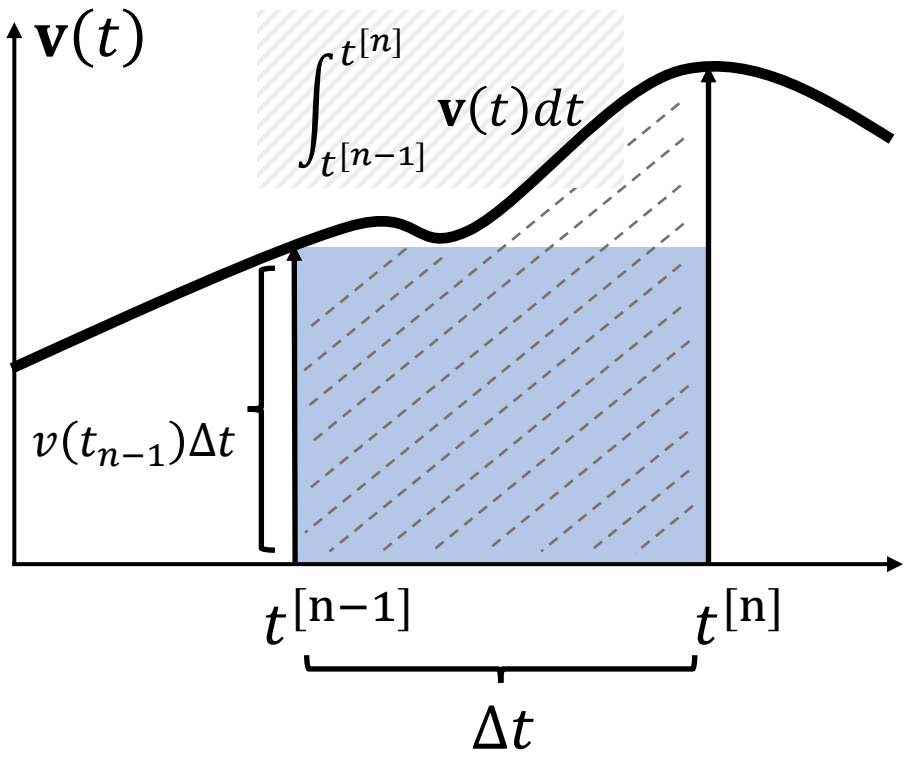
integrate

$$\mathbf{x}_\mathrm{p}(t_n) - \mathbf{x}_p(t_{n-1}) = \int_{t_{n-1}}^{t_n} \mathbf{v}_\mathrm{p}(t)\, dt$$

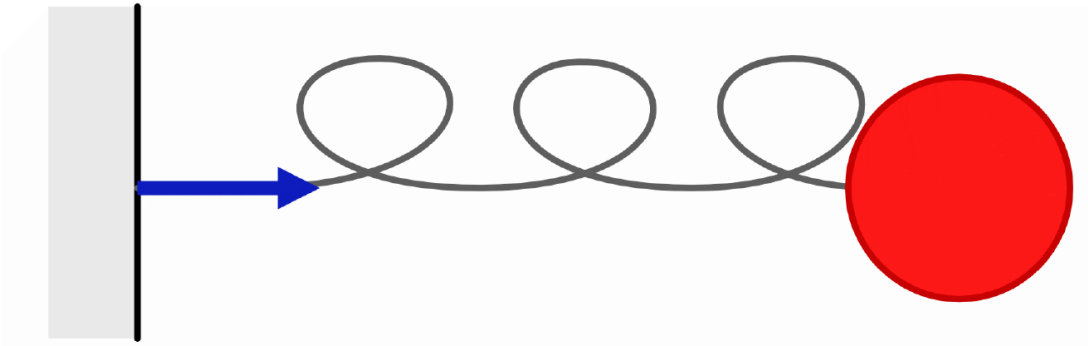
$$\mathbf{v}_p(t_n) - \mathbf{v}_\mathrm{p}(t_{n-1}) = \frac{1}{m}\int_{t_{n-1}}^{t_n} f(\mathbf{x}_p, t)\, dt$$

Temporal Discretization: Explicit Euler

- We don't want integrations
- $\mathbf{x}(t_n) - \mathbf{x}(t_{n-1}) = \int_{t_{n-1}}^{t_n} \mathbf{v}\,(t)\,dt \approx \mathbf{v}(t_{n-1})\Delta t$
- $\mathbf{v}(t_n) - \mathbf{v}(t_{n-1}) = \frac{1}{m}\int_{t_{n-1}}^{t_n} \mathbf{f}(t)\,dt \approx \frac{1}{m}\mathbf{f}(t_{n-1})\Delta \mathrm{t}$
- We know this is very inaccurate…

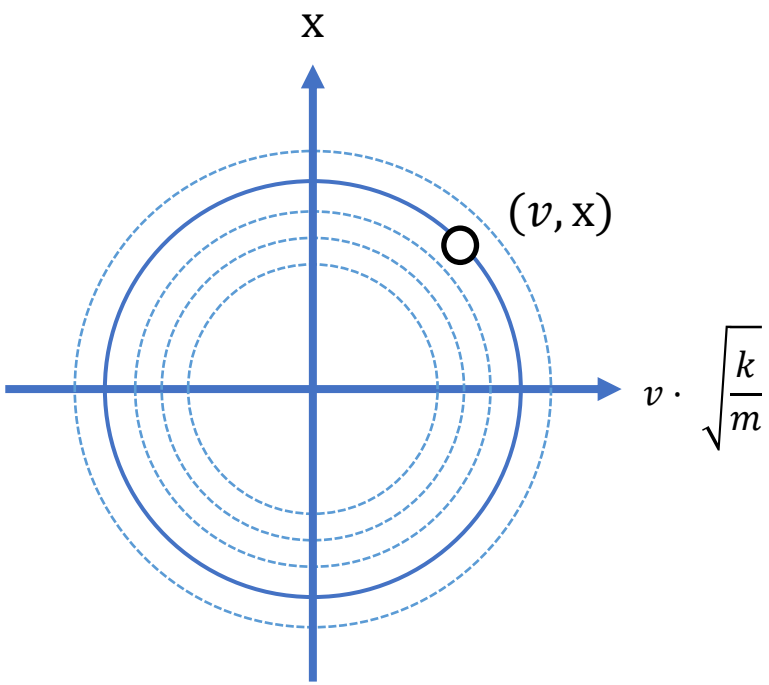


Time Discretization and Stability



$$m\frac{dv}{dt} = -k\mathrm{x}$$

$$\frac{d\mathrm{x}}{dt} = v$$



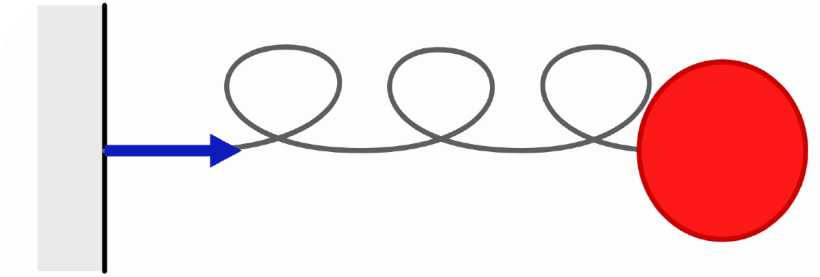
Rewrite in matrix form

$$\underbrace{\begin{pmatrix} m & 0 \\ 0 & 1 \end{pmatrix}}_{\mathrm{A}} \underbrace{\frac{d}{dt}\begin{pmatrix} v \\ \mathrm{x} \end{pmatrix}}_{\dot{\mathbf{y}}} = \overbrace{\begin{pmatrix} 0 & -k \\ 1 & 0 \end{pmatrix} \underbrace{\begin{pmatrix} v \\ \mathrm{x} \end{pmatrix}}_{\mathbf{y}}}^{\mathbf{f}(\mathbf{y})}$$

1st Order ODE

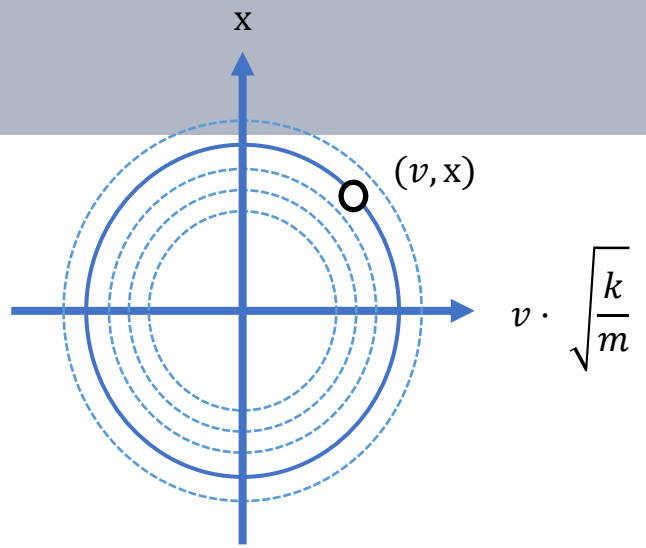
$$\mathrm{A}\dot{\mathbf{y}} = \mathbf{f}(\mathbf{y})$$

Stability



$$m\frac{dv}{dt} = -kx$$

$$\frac{dx}{dt} = v$$



Explicit Euler:

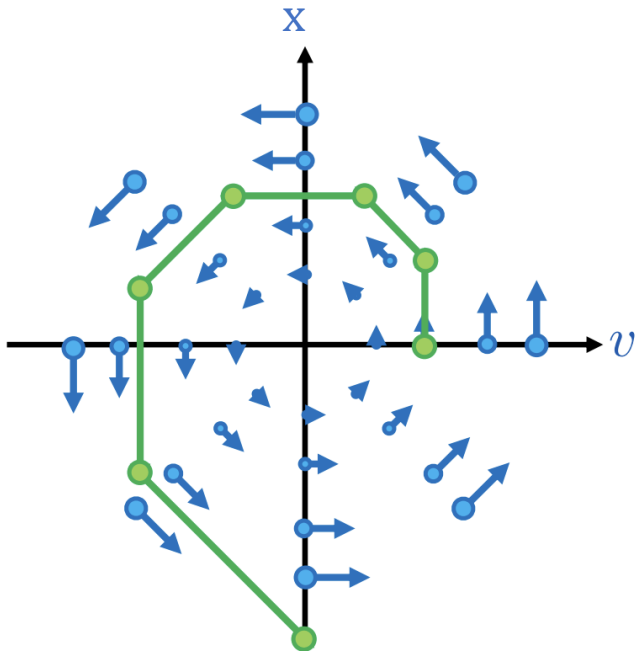
$$\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_n)\Delta t$$

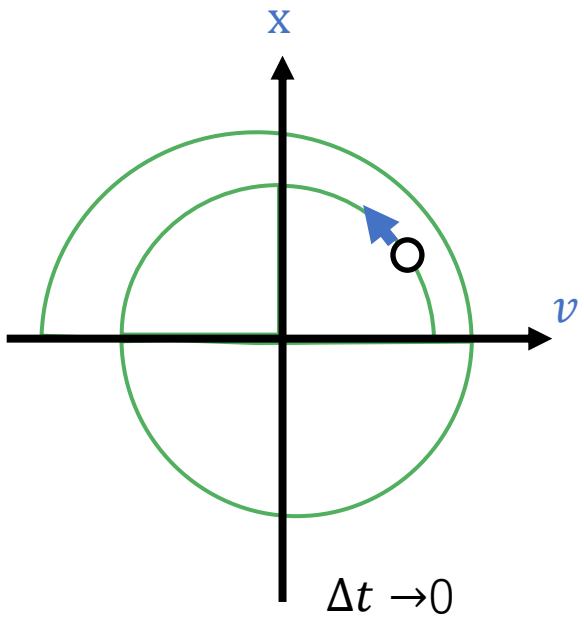
$$\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) - \frac{k}{m}\mathbf{x}(t_n)\Delta t$$

$$\mathbf{y} = \begin{pmatrix} v \\ x \end{pmatrix}, \quad \mathbf{y}(t_{n+1}) = \mathbf{y}(t_n) + \Delta t \begin{pmatrix} 0 & -\frac{k}{m} \\ 1 & 0 \end{pmatrix} \mathbf{y}(t_n)$$

$\Delta t \to 0$

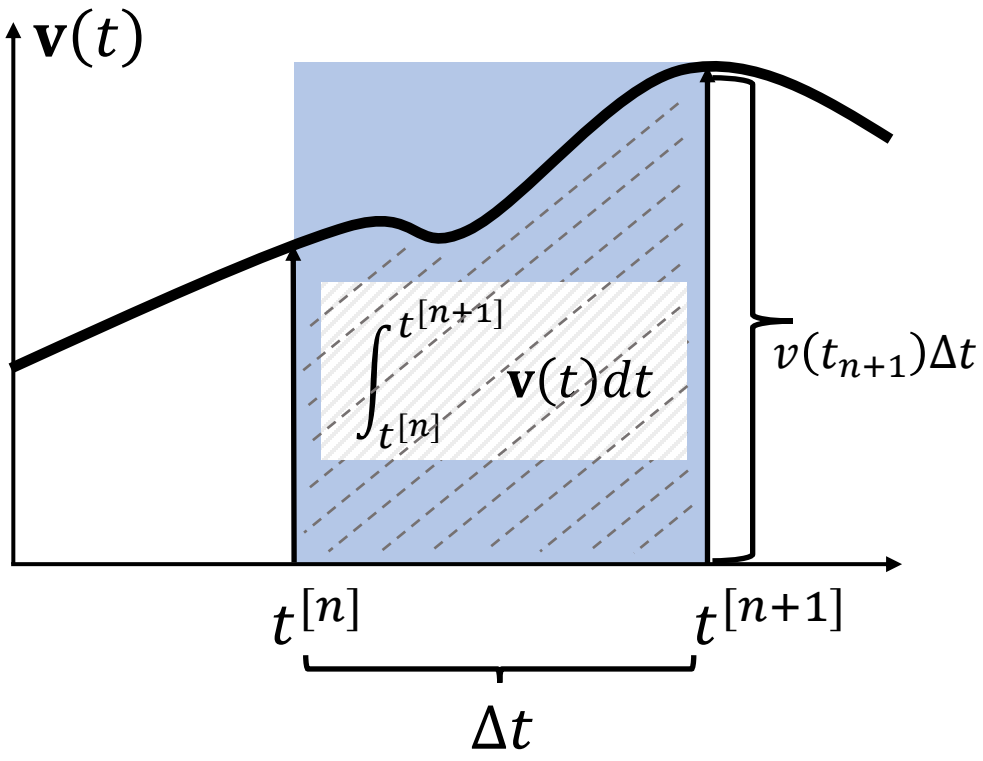
Unconditional Unstable for undamped springs!



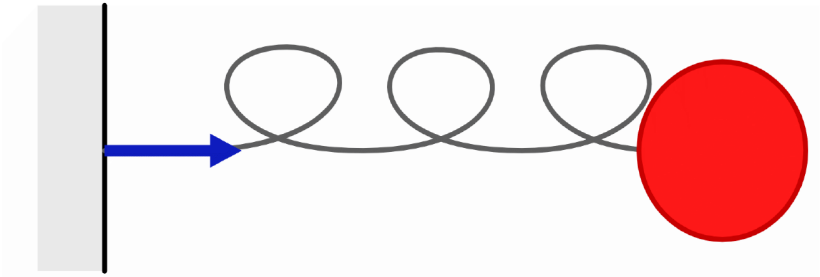


To Improve Stability: Implicit Euler

- Explicit Euler:
  - $\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_n)\Delta t$
  - $\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \frac{1}{m}\mathbf{f}(t_n)\Delta \mathrm{t}$
- Implicit Euler:
  - $\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_{n+1})\Delta t$
  - $\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \frac{1}{m}\mathbf{f}(t_{n+1})\Delta \mathrm{t}$
- Later we'll see why implicit Euler is stable

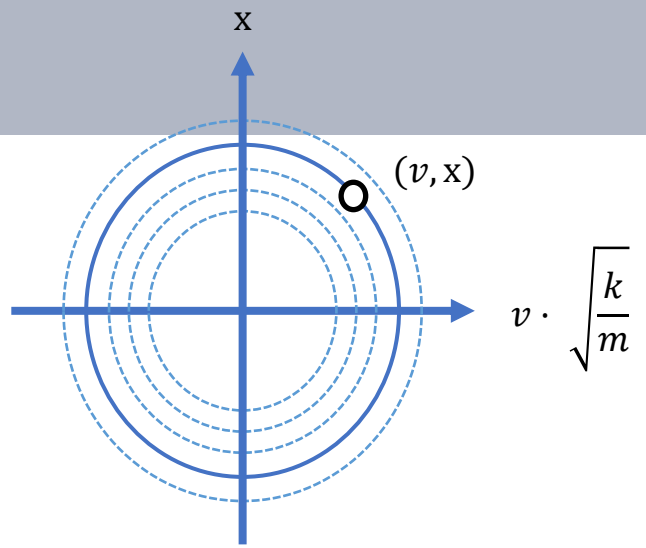


Stability



$$m\frac{dv}{dt} = -kq$$

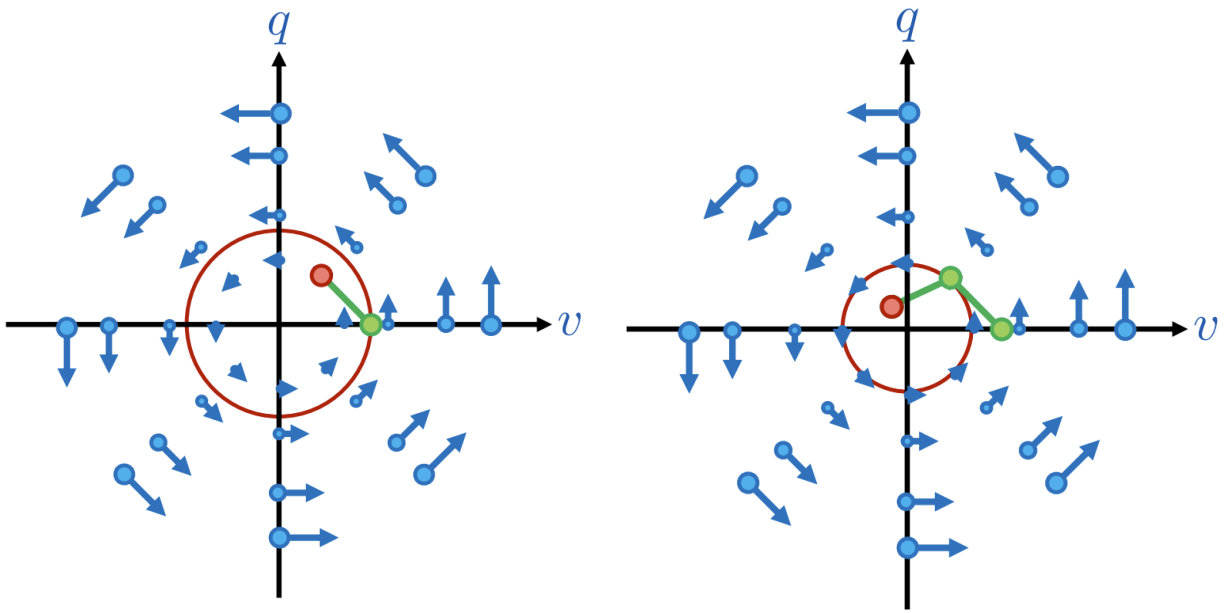
$$\frac{dq}{dt} = v$$



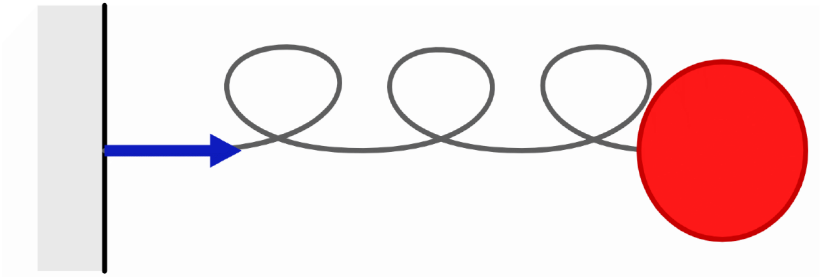
Implicit Euler:

$$\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_{n+1})\Delta t$$

$$\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \frac{1}{m}\mathbf{f}(t_{n+1})\Delta \mathrm{t}$$

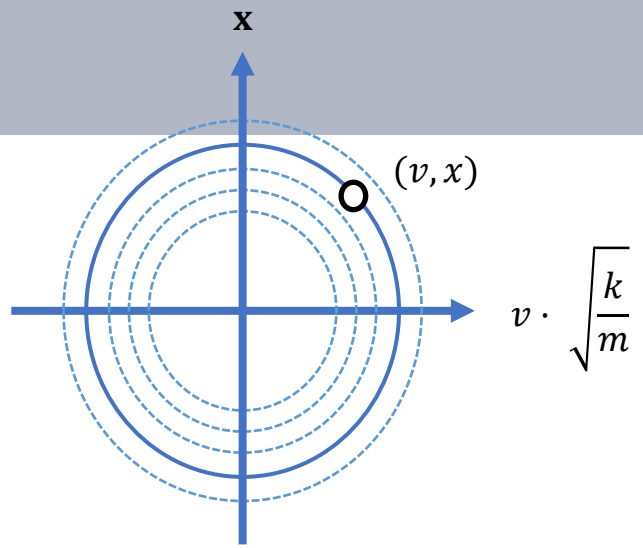


Stability



$$m\frac{dv}{dt} = -kx$$

$$\frac{dx}{dt} = v$$

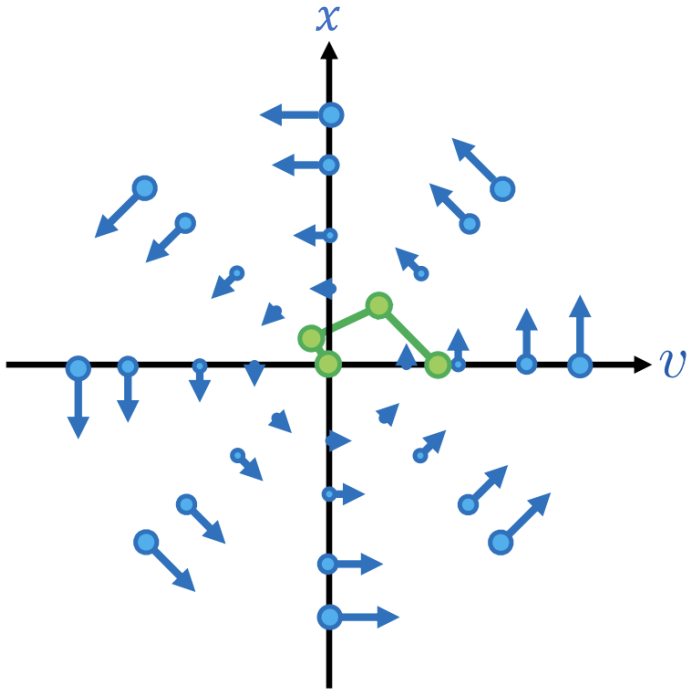


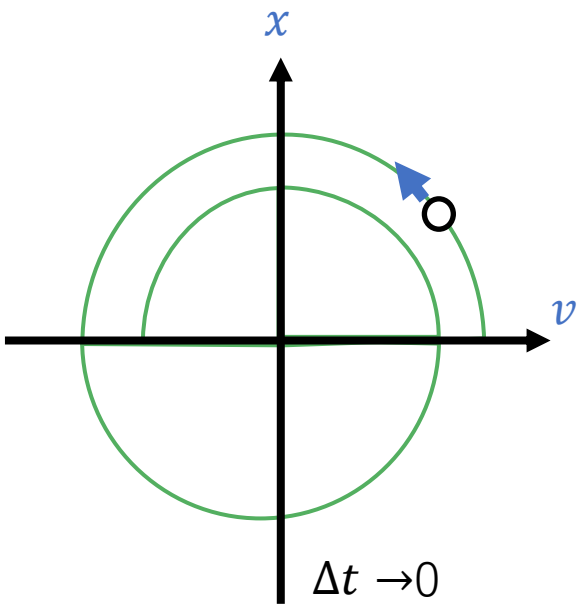
Implicit Euler:

$$\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_{n+1})\Delta t$$

$$\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \frac{1}{m}\mathbf{f}(t_{n+1})\Delta \mathrm{t}$$

Unconditional Stable for undamped springs!





Implicit Euler

$$\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_{n+1})\Delta t$$

$$\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \frac{1}{m}\mathbf{f}(t_{n+1})\Delta \mathrm{t}$$

$$\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_n)\Delta \mathrm{t} + \frac{1}{m}\mathbf{f}(t_{n+1})\Delta \mathrm{t}^2$$

$$\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_n)\Delta \mathrm{t} - \frac{1}{m}k\mathbf{x}(t_{n+1})\Delta \mathrm{t}^2$$

Implicit: Solve the numerical equation $\rightarrow$ Numerical Solver

Symplectic Euler (semi-explicit Euler)

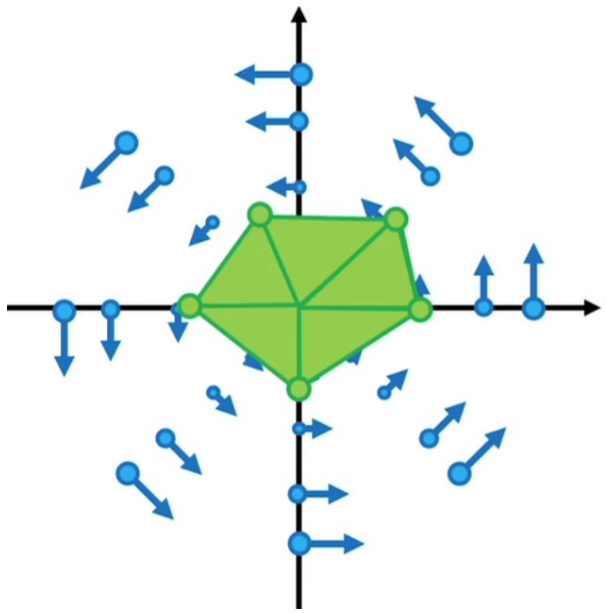
- First take an explicit velocity step,

$$\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \frac{1}{m}\mathbf{f}(t_n)\Delta \mathrm{t}$$

- Next take an implicit position step.

$$\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_{n+1})\Delta t$$

- Can ‘cancel out’ part of exploding and damping.



$$\begin{bmatrix} x_{n+1} \\ v_{n+1} \end{bmatrix} = \begin{bmatrix} 1-\omega^2\Delta t^2 & \Delta t \\ -\omega^2\Delta t & 1 \end{bmatrix} \begin{bmatrix} x_n \\ v_n \end{bmatrix}, \ \omega = \sqrt{\frac{k}{m}}$$

https://interactivecomputergraphics.github.io/physics-simulation/
https://interactivecomputergraphics.github.io/physics-simulation/examples/spring_plot.html
https://interactivecomputergraphics.github.io/physics-simulation/examples/time_integration.html

How to Evaluate Time Integration?

- Error?

  Discretization → Truncation Errors:

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2}h^2 + \dots$$

$O(h^{n+1})$ *Truncation Error*

n^{th} *Order Approximation*

- Stability:

  Do *errors* accumulate? (Error Propagation, Energy)

- Convergence (Consistency):

  If $h \to 0$, will $Error \to 0$?

- Accuracy and Convergence (Speed):

  n^{th} *Order Accurate* with $O(h^{n+1})$ error

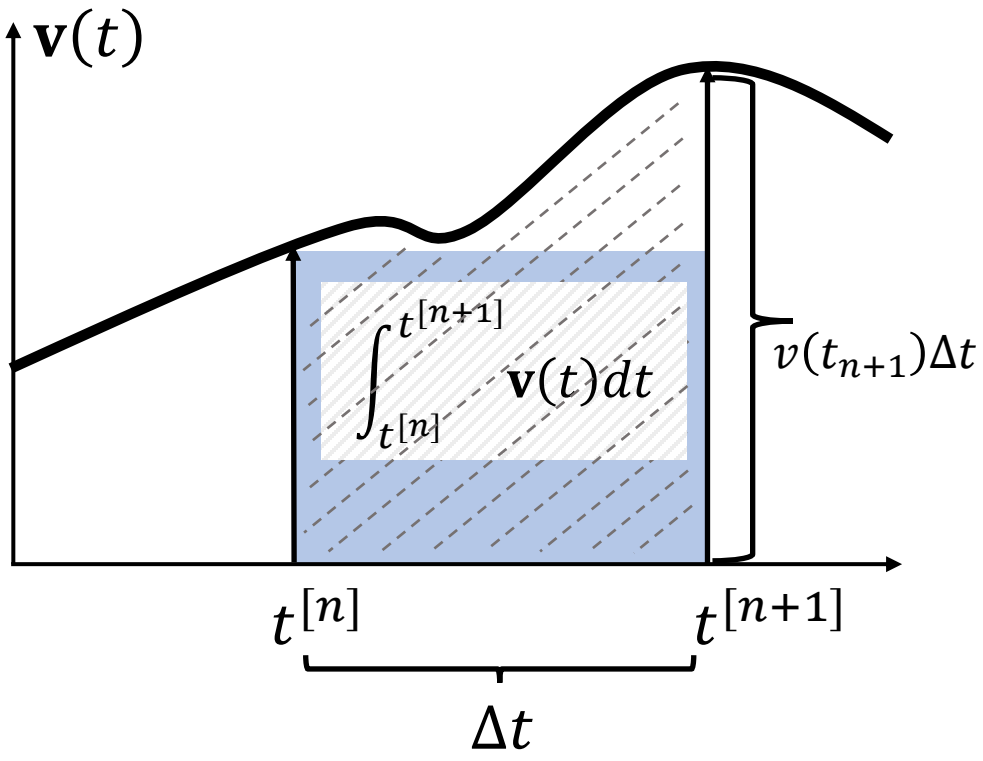
To Improve Accuracy

- Explicit Euler:
  - $\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_n)\Delta t$
  - $\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \frac{1}{m}\mathbf{f}(t_n)\Delta \mathrm{t}$
- Truncation Errors:
  - $\mathbf{x}(t_{n+1}) - [\mathbf{x}(t_n) + \int_{t_n}^{t_{n+1}} \mathbf{v}(t)dt\,]$

$$= \mathbf{v}(t_n)\Delta t - \left[\Delta t\mathbf{v}(t_n) + \frac{\Delta t^2}{2}\mathbf{v}'(t_n) + \cdots\right]$$

$$= O(\Delta t^2)$$

- 1st-order accurate

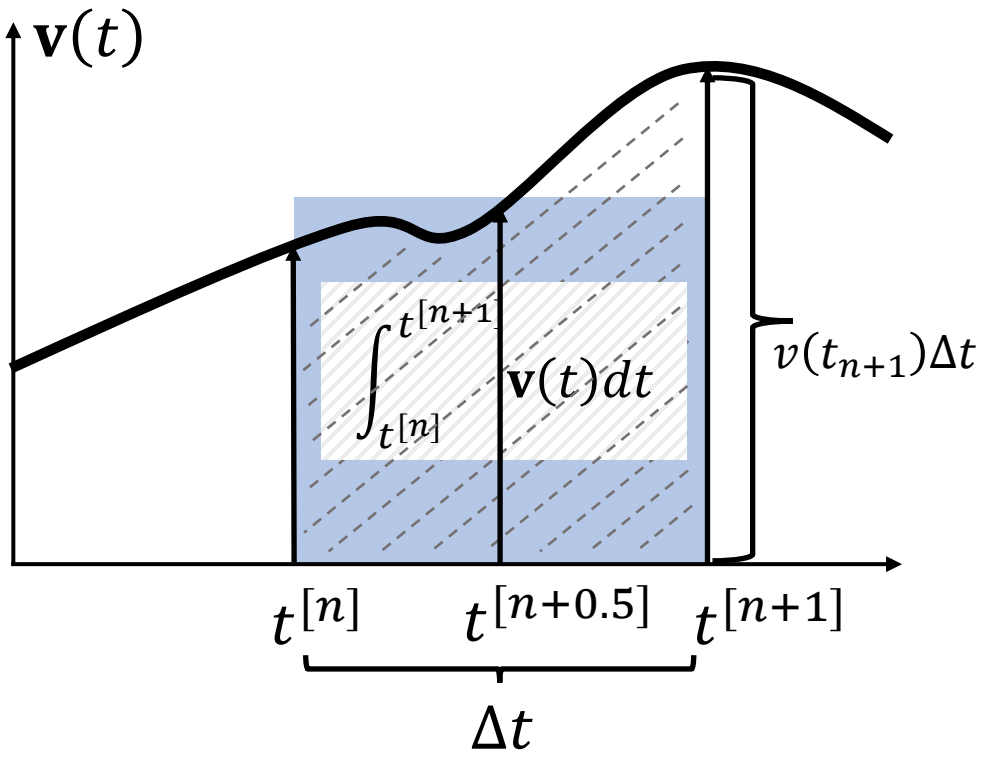


To Improve Accuracy: Mid-Point Method

- Mid-Point Method:
  - $\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \mathbf{v}(t_{n+0.5})\Delta t$
  - $\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \frac{1}{m}\mathbf{f}(t_{n+0.5})\Delta \mathrm{t}$
- Explicit Mid-Point Method:
  1. **Calculate $\mathbf{v}(\mathrm{t}_{\mathrm{n+0.5}}), \mathbf{f}(\mathrm{t}_{\mathrm{n+0.5}})$, by a half Euler Step (with $\Delta \mathrm{t}/2$)**
  2. **Calculate $\mathbf{x}(t_{n+1})$, $\mathbf{v}(t_{n+1})$, using a full step (similar to an Euler step)**
  - $\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + [\mathbf{v}(t_n) + \frac{\Delta \mathrm{t}}{2}\frac{\mathbf{f}(t_n)}{m}]\Delta t$
- Truncation Errors:
  - $\mathbf{x}(t_{n+1}) - [\mathbf{x}(t_n) + \int_{t_n}^{t_{n+1}} \mathbf{v}(t)dt\ ]$

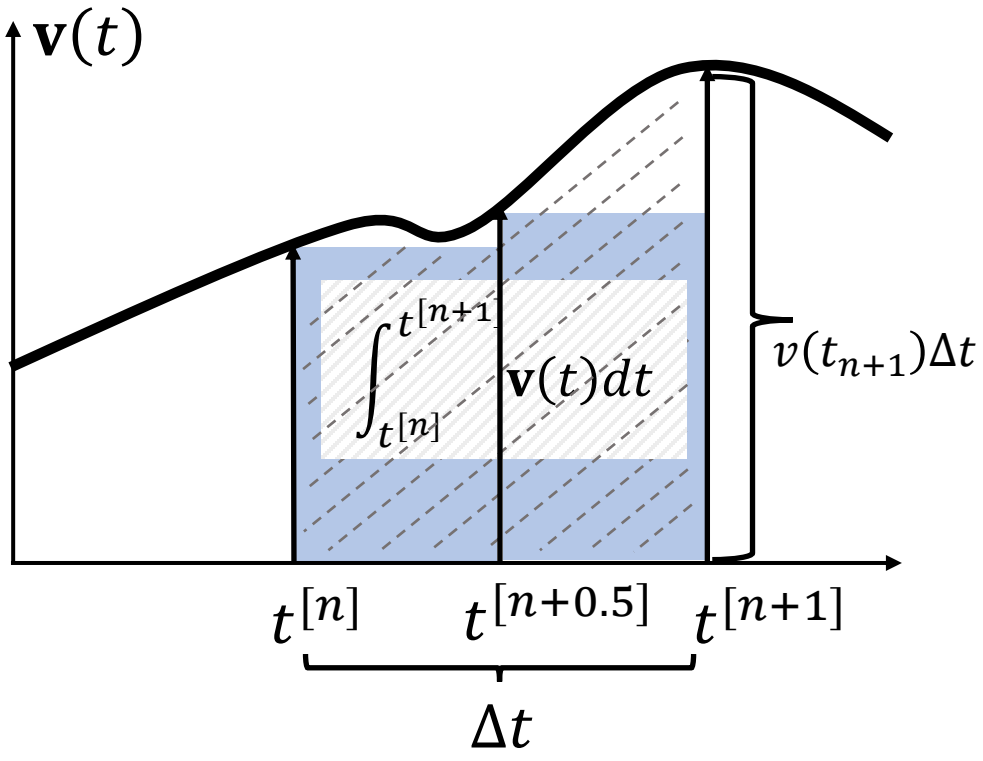
    $= [\mathbf{v}(t_n) + \frac{\Delta t}{2}\mathbf{v}'(t_n)]\Delta t - \left[\Delta t\mathbf{v}(t_n) + \frac{\Delta t^2}{2}\mathbf{v}'(t_n) + \cdots\right]$

    $= O(\Delta t^3)$
- 2st-order accurate

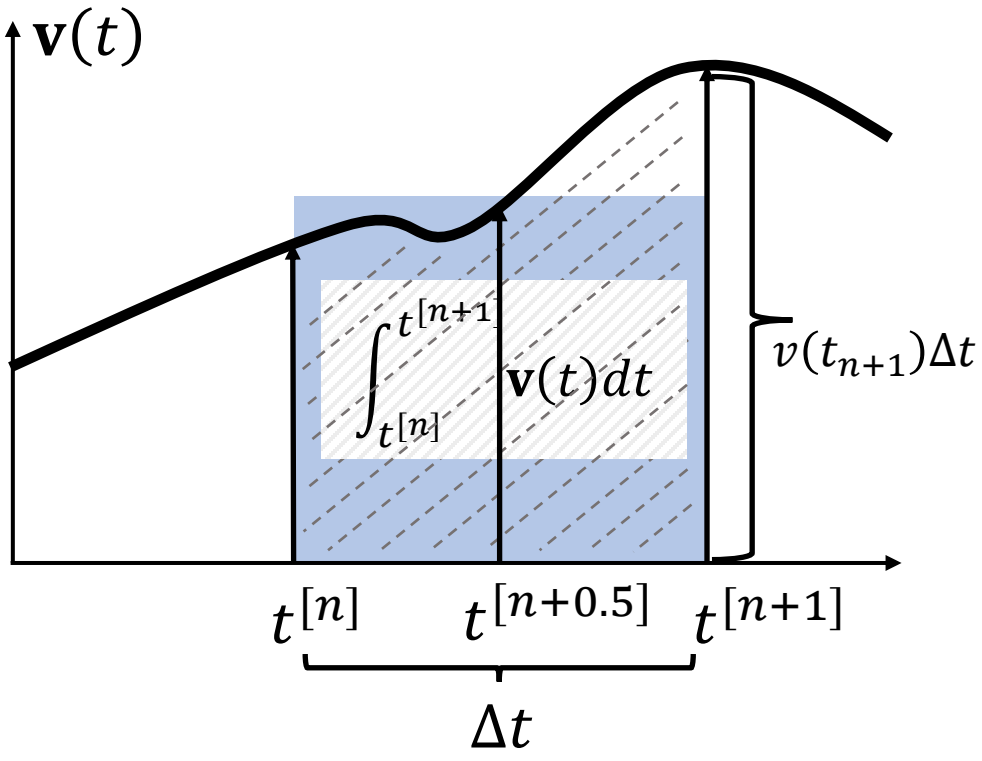


To Improve Accuracy: Mid-Point Method

- Sub-step Euler (left) and Mid-Point Method (right)



N Sub-step Euler Error: $O(\Delta t^2/n^2)$



Mid-Point Method Error: $O(\Delta t^3)$

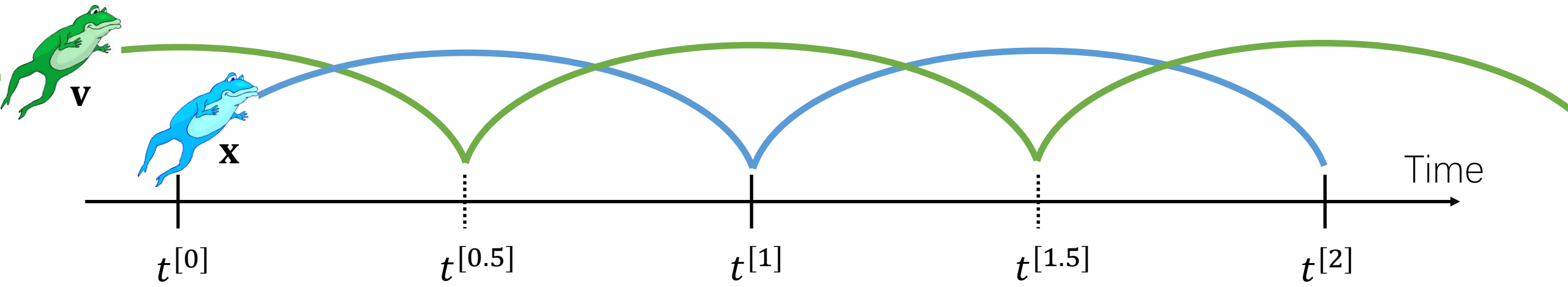
Error with n sub steps of a 1st order method is usually much higher than n-th order method

To Improve Accuracy: Runge-Kutta methods

- Mid-Point Method: A second-order Runge-Kutta method
  - $\mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \Delta t \mathbf{v}(t_{n+0.5})$; $\mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \frac{1}{m}\mathbf{f}(t_{n+0.5})\Delta \mathrm{t}$
- Explicit RK4 with $4th\ Order$ Accuracy
  - $$\begin{cases} \mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \Delta t\left[\frac{1}{6}\dot{\mathbf{x}}_1 + \frac{1}{3}\dot{\mathbf{x}}_2 + \frac{1}{3}\dot{\mathbf{x}}_3 + \frac{1}{6}\dot{\mathbf{x}}_4\right] \\ \mathbf{v}(t_{n+1}) = \mathbf{v}(t_n) + \Delta t[\frac{1}{6}\dot{\mathbf{v}}_1 + \frac{1}{3}\dot{\mathbf{v}}_2 + \frac{1}{3}\dot{\mathbf{v}}_3 + \frac{1}{6}\dot{\mathbf{v}}_4] \end{cases}$$
  - $\dot{\mathbf{x}}_1 = \mathbf{v}(t_n)$; $\dot{\mathbf{v}}_1 = \frac{1}{m}f(\mathbf{x}(t_n), \mathbf{v}(t_n))$
  - $\dot{\mathbf{x}}_2 = \mathbf{v}(t_n) + \frac{\Delta t}{2}\dot{\mathbf{v}}_1$; $\dot{\mathbf{v}}_2 = \frac{1}{m}f(\mathbf{x}(t_n) + \frac{\Delta t}{2}\dot{\mathbf{x}}_1, \mathbf{v}(t_n) + \frac{\Delta t}{2}\dot{\mathbf{v}}_1)$
  - $\dot{\mathbf{x}}_3 = \mathbf{v}(t_n) + \frac{\Delta t}{2}\dot{\mathbf{v}}_2$; $\dot{\mathbf{v}}_3 = \frac{1}{m}f(\mathbf{x}(t_n) + \frac{\Delta t}{2}\dot{\mathbf{x}}_2, \mathbf{v}(t_n) + \frac{\Delta t}{2}\dot{\mathbf{v}}_2)$
  - $\dot{\mathbf{x}}_4 = \mathbf{v}(t_n) + \Delta t\dot{\mathbf{v}}_3$; $\dot{\mathbf{v}}_4 = \frac{1}{m}f(\mathbf{x}(t_n) + \Delta t\dot{\mathbf{x}}_3, \mathbf{v}(t_n) + \Delta t\dot{\mathbf{v}}_3)$

To Improve Accuracy: Leap-Frog Integrator

- Interleaved update of velocity and position
- just one force evaluation, but 2nd order accuracy
- $\begin{cases} \mathbf{v}(t_{n+0.5}) = \mathbf{v}(t_{n-0.5}) + \boldsymbol{a}(t_n)\Delta \mathrm{t} \\ \mathbf{x}(t_{n+1}) = \mathbf{x}(t_n) + \Delta t\ \mathbf{v}(t_{n+0.5}) \end{cases}$ ← Mid-point ← Mid-point



How to Evaluate Time Integration?

- Error?

  Discretization → Truncation Errors:

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2}h^2 + \dots$$

$O(h^{n+1})$ *Truncation Error*

n^{th} *Order Approximation*

- Stability:

  Do *errors* accumulate? (Error Propagation, Energy)

- Convergence (Consistency):

  If $h \to 0$, will $Error \to 0$?

- Accuracy and Convergence (Speed):

  n^{th} *Order Accurate* with $O(h^{n+1})$ error

Numerical Solver

To solve matrix equations (linear equation systems, e.g., pressure solve for fluids):

- Jacobi/Gauss-Seidel Iteration
- Conjugate Gradient
- Multigrid…

Other solvers for nonlinear equation systems include:

- Newton's Method (e.g., implicit Euler for soft body dynamics)
- Quasi-Newton Methods
- BFGS…

No one general optimal solver for all problems

Conclusions

- Three Building Blocks
- Spatial Discretization:
  - Lagrangian and Eulerian Viewpoints
  - ODEs and PDE problems
  - Particles, Mesh, Grid
  - Vector and Tensor Calculus
- Time Discretization
  - Explicit, Implicit, Symplectic Euler
  - Mid-Point Method, Runge-Kutta methods, Leap-Frog Integrator
  - Evaluation: Convergence, Stability, Accuracy
- Numerical Solvers